NAZARBAYEV UNIVERSITY

Programming for Engineers

Detailed Reading Notes

Week 5: Tuples, Lists, and Ordered Engineering Data

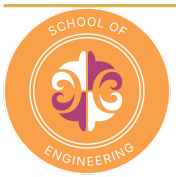
Yanwei Wang

School of Engineering

Nazarbayev University

Fall 2026

Store ⟶ Access ⟶ Change deliberately ⟶ Preserve meaning and units



About these notes

These notes accompany *Programming for Engineers: Week 5 — Tuples, Lists, and Ordered Engineering Data*, the 81-page lecture presentation by Yanwei Wang for Fall 2026. They follow Lecture 13, Lecture 14, and Lecture 15 in the same order as the slides. The purpose is to turn each short slide into an explanation that you can read, run, trace, and revisit independently.

Course information

Course: ENG 101 — Programming for Engineers, lecture section 2L, 6 ECTS credits.
Instructor: Dr. Yanwei Wang, School of Engineering, Nazarbayev University.
Term: Fall 2026.
Week 5 focus: lists, tuples, indexing, slicing, list operations, mutability, copying, structured records, and preparation for Laboratory 2.

How the slides and notes relate. The main examples retain the slides' data, terminology, and engineering interpretation. Explanations and worked activity answers are developed here from those examples. Each section identifies the relevant *PDF page numbers*, counting the title and lecture-divider pages. These differ from some numbers printed in slide footers. A complete slide-to-notes map appears at the end.

Code you can study. Every Python listing contains comments. A comment beginning with `##` explains meaning to the reader; it is not an executable instruction. The displayed line numbers are reference aids and must not be typed into Python. Code listings use a gold background; output uses grey. Working examples are available as separate `.py` files in the source package and as a single companion script. Listings marked *deliberate error* must be run separately: they demonstrate an exception rather than a successful calculation.

Technical clarifications. Short slides sometimes rely on a restricted example. Clearly labelled clarification boxes state those restrictions explicitly, especially for slice length, empty data, shallow copying, and tuple immutability. Visible typesetting fragments such as "exttt" in a few slide headings are rendered as normal code names here. Display-wrapped strings are written as valid Python lines. These are presentation repairs, not new programming rules.

Laboratory note

The slides describe a *proposed* Laboratory 2 theme, not the final assessed brief. The example dataset is preserved exactly. The later worked report labels its warning threshold and calibration offset as *illustrative assumptions*; the slides do not supply those values. Moodle remains the authority for the final task, deadline, submission requirements, collaboration rules, and permitted resources.

How to study Week 5

A list is easy to type but easy to misunderstand. The important questions are not only "What is printed?" but also "Which object changed?", "Which sample does this position represent?", and "Can I still recover the original measurements?" Use the following sequence for every example.

1. **Identify the representation.** State whether the object is a list, a tuple, one element, or a collection of records. Write the quantity, unit, and meaning of the order.
2. **Predict before running.** For indexing and slicing, mark the selected indices. For a mutation, draw the entire list before and after the statement.
3. **Separate two questions.** Does the operation change an existing object? What value does the operation return?
4. **Run from a known state.** Recreate the starting data before comparing two versions. Re-running a calibration cell on already corrected data can apply the correction twice.
5. **Test a boundary.** Try an empty list, one element, an absent value, repeated values, or a reading exactly at a stated limit.
6. **Explain the engineering meaning.** A correct numerical summary is not a substitute for the original series, its units, its measurement quality, or its acquisition order.

Lecture	Main question	Study emphasis
13	How do I access ordered data?	Lists, indices, slices, summaries, value loops, index loops, and `enumerate()`.
14	How do I change data deliberately?	Replacement, building and filtering, insertion and removal, sorting, reversing, aliasing, copying, and function side effects.
15	How do I keep related values together?	Tuples, immutability, packing and unpacking, paired records, interval velocity, RMS, and the Laboratory 2 theme.

Scope and outcomes

Each lecture states its own learning outcomes. By the end of the week, you should be able to access, process, change, and explain ordered engineering collections while preserving their meaning and units. This week uses native Python lists and tuples; dictionaries, recursion, and scientific-computing libraries are outside the attached slide set.

Key idea

Store $\longrightarrow$ Access $\longrightarrow$ Change deliberately $\longrightarrow$ Preserve meaning and units. This is the organizing message of the Week 5 slides. Keep all four steps visible when explaining a program.

Contents

About these notes 2

How to study Week 5 3

1. Lecture 13: Ordered Collections, Indexing, and Slicing 6
- 1.1. One sensor, several measurements . . . 6
- 1.2. Creating and inspecting a list . . . 7
- 1.3. Order, position, and value . . . 8
- 1.4. Diagnosing an invalid index . . . 9
- 1.5. Slicing retrieves a data window . . . 9
- 1.6. Open-ended slices, steps, and reverse order . . . 11
- 1.7. Prediction activity: index or slice? . . . 11
- 1.8. Length and membership answer different questions . . . 12
- 1.9. Mean, minimum, maximum, and range . . . 13
- 1.10. Loop directly over values . . . 14
- 1.11. Loop over indices when position matters . . . 15
- 1.12. Use enumerate() for index and value together . . . 15
- 1.13. Trace the first threshold crossing . . . 16
- 1.14. Preserve quantity, units, and acquisition order . . . 17
- 1.15. Debugging clinic: three different misconceptions . . . 17
- 1.16. Lecture 13 exit ticket and summary . . . 18

2. Lecture 14: List Mutability and List Operations 19
- 2.1. Warm-up: trace every change . . . 19
- 2.2. Mutable means the existing object can change . . . 20
- 2.3. Calibration by changing each indexed element . . . 20
- 2.4. Safer calibration: build a processed list . . . 21
- 2.5. Building and filtering with append() . . . 22
- 2.6. Append one object; extend by its elements . . . 23
- 2.7. Insert at a chosen position . . . 24
- 2.8. Remove by value or remove by position . . . 24
- 2.9. Why deleting during iteration can skip values . . . 25
- 2.10. Safe filtering into a new list . . . 26
- 2.11. Concatenation and repetition are not vector arithmetic . . . 26
- 2.12. Two questions for every list operation . . . 27
- 2.13. Sorting: change the original or keep a ranked copy . . . 28
- 2.14. Reversal: mutate or preserve . . . 29
- 2.15. Aliasing: two names, one list . . . 29
- 2.16. A separate copy, and the meaning of shallow . . . 30
- 2.17. Functions can mutate the caller's list . . . 31
- 2.18. A traceable data-processing workflow . . . 33
- 2.19. Lecture 14 exit ticket and summary . . . 33

3. Lecture 15: Tuples, Structured Records, and Laboratory 2 34
- 3.1. Warm-up: which structure should change? . . . 34
- 3.2. A tuple is an immutable ordered sequence . . . 34
- 3.3. Empty, singleton, and multi-element tuples . . . 35
- 3.4. Familiar sequence operations transfer to tuples . . . 36

3.5. Immutability prevents item assignment . . . 36
3.6. Packing and unpacking . . . 37
3.7. Multiple assignment can swap two values . . . 38
3.8. Functions return a tuple of related results . . . 39
3.9. Document the meaning of each tuple position . . . 39
3.10. A list of tuples stores paired data . . . 40
3.11. Unpack records directly in a loop . . . 41
3.12. Parallel lists versus paired records . . . 41
3.13. Interval velocity from adjacent records . . . 42
3.14. Compare the shared interface and the mutation constraint . . . 43
3.15. Convert only when the representation requirement changes . . . 43
3.16. Structure-selection activity: justify the intended use . . . 44
3.17. Laboratory 2: source dataset and proposed tasks . . . 45
3.18. RMS: square, average, then take the square root . . . 45
3.19. Worked study report for all ten readings . . . 47
3.20. Hand-checkable tests for the laboratory pattern . . . 50
3.21. Debugging clinic: tuple and record mistakes . . . 52
3.22. Concept map, exit ticket, and Lecture 15 summary . . . 54

4. Week 5 Synthesis, Reference, and Worked Practice 55
4.1. Read a collection program as an engineering argument . . . 55
4.2. Quick reference: sequence access and iteration . . . 55
4.3. Quick reference: operations and tuple patterns . . . 56
4.4. A working glossary . . . 56
4.5. Cumulative practice questions . . . 58
4.6. Worked solutions and explanations . . . 59
4.7. A readiness checklist for the next week . . . 62

A. Source Alignment and Technical Clarifications 63
A.1. Primary source and recommended reading . . . 63
A.2. What has been clarified or added . . . 63
A.3. Slide-to-notes coverage map . . . 64
A.4. Using the accompanying Python files . . . 66

1 Lecture 13: Ordered Collections, Indexing, and Slicing

Lecture-slide alignment: Week 5 source PDF, pp. 3–27. PDF page numbers include title and divider pages.

Learning outcomes

By the end of this chapter, you should be able to explain why engineers need ordered collections; create and inspect a list; distinguish indices from values; use forward and negative indexing; diagnose invalid indices; extract data windows using slices; use `len()`, `sum()`, `min()`, and `max()`; and select a loop that preserves the relationship between values, positions, and physical meaning.

1.1. One sensor, several measurements

Lecture-slide alignment: Week 5 source PDF, pp. 5–6. PDF page numbers include title and divider pages.

The opening slide describes a temperature sensor that records once per second. At the five times below, the sensor produces five different values.

Time (s)	0	1	2	3	4
Temperature (°C)	22.1	22.5	23.0	23.4	22.9

A scalar variable can hold one reading at a particular moment. Reassigning that name after each measurement does not preserve the complete sequence. Five unrelated variables could store the five numbers, but they would make looping, adding measurements, and selecting a window unnecessarily awkward. A **collection** makes the relationship explicit: these values belong to one series.

The word **ordered** matters. The third measurement was recorded after the second and before the fourth. Rearranging the numbers can preserve their mean and maximum while destroying that chronology. The time of a peak, the first threshold crossing, and the direction of a trend depend on order, not merely on which numbers exist.

Warm-up discussion: possible answers

One scalar cannot represent the entire retained history by itself. A list can preserve the acquisition sequence. Useful summaries might include the mean, minimum, maximum, range, or count above a stated limit. However, the question determines which summary is useful: a mean alone cannot identify when the highest temperature occurred.

The same representation issue appears with candidate dimensions, forces recorded during a test, simulation states, or measurements from several sensors. Before coding, state what one element represents and what moving from one index to the next means.

1.2. Creating and inspecting a list

Lecture-slide alignment: Week 5 source PDF, p. 7. PDF page numbers include title and divider pages.

A Python **list** is an ordered, mutable sequence of objects. At this stage, think of it as an ordered row of places, each of which contains a reference to an element. Square brackets delimit the list; commas separate the elements. The name before = is a variable referring to the entire list, not just its first element.

```
# Store temperatures in acquisition order; every element is in deg C.
temperature_readings_c = [
    22.1, 22.5, 23.0, 23.4, 22.9
]
# Display the complete list, including brackets and commas.
print(temperature_readings_c)
# Confirm that this object has Python's list type.
print(type(temperature_readings_c))
```

Code 1.1: A first list of temperature readings (source PDF p. 7).

Runnable example: `ex01_first_list.py`

```
[22.1, 22.5, 23.0, 23.4, 22.9]
<class 'list'>
```

The opening bracket allows the expression to continue across several source lines. The closing bracket completes the list expression, after which the assignment binds `temperature_readings_c` to the list. `print()` displays the collection; `type()` reports its type. Neither call changes its contents.

1.2.1. Empty, singleton, and mixed lists

An empty list is written `[]`. It is not the same as `[0]`, which contains one value, or `None`, which is a different kind of object. A one-element list needs no special trailing comma: `[22.1]` is already a list because of its square brackets.

```
# An empty list has no elements; it can receive values later.
empty_readings = []
# A singleton list contains one measurement in deg C.
single_reading_c = [22.1]
# Lists can store text labels as well as numerical quantities.
sensor_labels = ["T-01", "T-02"]
# Python permits mixed types; document the positions and units.
mixed_record = ["T-01", 22.1, "deg C"]
# Count elements, not printed characters or bytes.
print(len(empty_readings), len(single_reading_c))
```

Code 1.2: Empty, singleton, and mixed lists: explanatory examples.

Runnable example: `ex02_list_shapes.py`

A list may contain numbers, strings, or other objects. Python permitting mixed types does not mean that every numerical operation is appropriate. The mixed record above stores a label, a reading, and a unit; adding those fields with `sum()` would not make engineering sense. Numerical summaries in this chapter assume that the elements are compatible numerical values in one unit.

Engineering check

Use `temperature_readings_c` rather than a name that hides the quantity. A list of plain numbers does not automatically record that its values are temperatures, which sensor supplied them, or that the spacing between samples was one second.

1.3. Order, position, and value

Lecture-slide alignment: Week 5 source PDF, pp. 8–12. PDF page numbers include title and divider pages.

For the five-temperature list, the index diagram is:

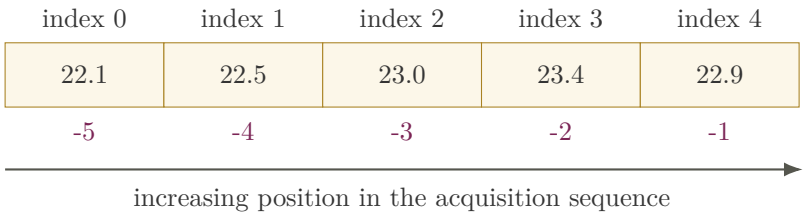


Figure 1.1.: The same five positions can be addressed from the beginning or from the end. Adapted from the ordered-list diagram on source PDF p. 8.

An **index** answers "Where is the element?" A **value** answers "What is stored at that position?" In `readings[2]`, the index is the integer `2`; the selected value is the temperature `23.0`. Index 2 is the third position because Python starts counting positions at zero.

```
# Store five temperatures in their original order (deg C).
readings = [22.1, 22.5, 23.0, 23.4, 22.9]
first = readings[0]       # Index 0 selects the first element.
second = readings[1]      # Index 1 selects the second element.
fourth = readings[3]      # Index 3 selects the fourth element.
print(first, second, fourth)  # Display the selected values.
last = readings[-1]       # Index -1 selects the final element.
second_last = readings[-2]  # Index -2 selects the previous element.
print(last, second_last)  # Neither indexing operation changes the list.
```

Code 1.3: Read individual values by index (source PDF pp. 9–11).

Runnable example: `ex03_indexing.py`

```
22.1 22.5 23.4
22.9 23.4
```

The expressions in square brackets perform indexing. In the assignment `first = readings[0]`, Python retrieves the element and binds the name `first` to that element. It does not remove the element from the list.

1.3.1. Forward and negative indices

For a nonempty list of length n, the valid forward indices are

$$0, 1, \ldots, n-1,$$

and the valid negative indices are

$$-1, -2, \ldots, -n.$$

The slides call the first group "positive indices"; more precisely, they are *nonnegative* or forward indices, because the first one is zero. A negative index j refers to the same position as $n + j$. With five readings, index -2 therefore corresponds to index $5 - 2 = 3$.

`x[-1]` is useful for the latest measurement even when the list length changes. But a negative sign does not make an index automatically valid: index -6 is invalid for a five-element list. An empty list has no valid element index at all, including `0` and `-1`.

Common mistake

`len(values)` is the number of elements, not the last valid forward index. If a list contains n elements, asking for `values[n]` asks for an $(n + 1)$st element that does not exist.

1.4. Diagnosing an invalid index

Lecture-slide alignment: Week 5 source PDF, p. 13. PDF page numbers include title and divider pages.

```
# There are three values; valid forward indices are 0, 1, and 2.
values = [10, 20, 30]
print(len(values))  # Prints 3, the count of elements.
# DELIBERATE ERROR: index 3 would require a fourth element.
print(values[3])
```

Code 1.4: Deliberate error: an index equal to the length (source PDF p. 13).

Runnable example: `ex04_bad_index.py`

```
3
IndexError: list index out of range
```

The length is printed successfully before the error. The final line then raises `IndexError` because the only valid forward positions are 0, 1, and 2. The exact traceback also contains file and line information that depends on where you run the example.

A disciplined repair starts with the intention. To retrieve the third element, use index 2. To retrieve the last element without assuming the length, use `values[-1]` after ensuring the list is nonempty. To add a fourth element, use an appropriate list-building operation rather than indexing a nonexistent location. These are different tasks; choosing a valid expression is not enough unless it expresses the intended one.

1.5. Slicing retrieves a data window

Lecture-slide alignment: Week 5 source PDF, pp. 14–15. PDF page numbers include title and divider pages.

Indexing retrieves one element. **Slicing** retrieves a sequence of selected elements. The pattern `x[start:stop]` includes the start position and excludes the stop position. The interval is often called **half-open**.

```python
# Store the original temperature series in deg C.
readings = [22.1, 22.5, 23.0, 23.4, 22.9]
# Include indices 1, 2, and 3; exclude stop index 4.
window = readings[1:4]
# A slice of a list is itself a list, even if it has one element.
print(window)
```

Code 1.5: Extract a data window (source PDF p. 14).

Runnable example: `ex05_window.py`

```
[22.5, 23.0, 23.4]
```

Indices 1, 2, and 3 are selected; index 4 is excluded. The returned object is a list, and the original list is not changed. Even `readings[2:3]` is a one-element list, whereas `readings[2]` is the element itself. This difference matters when deciding whether a later operation expects one number or a collection.

1.5.1. Why exclude the stop?

For in-range forward bounds with $0 \leq a \leq b \leq n$ and step 1, the number of selected elements is $b - a$. Adjacent windows `x[0:3]` and `x[3:6]` meet at index 3 without selecting it twice. They can partition an available six-element series into two non-overlapping windows.

Clarification of the slice-length statement on PDF p. 15

The short rule "length is $b - a$" assumes step 1 and valid forward bounds in increasing order. Python clips oversized slice bounds, interprets negative bounds relative to the end, and can return an empty slice. The formula is not unconditional. The examples below make the omitted conditions visible.

```python
# Use a short list so every available index is visible.
values = [10, 20, 30]
print(values[1:3])    # [20, 30]: in-range bounds give length 3 - 1.
print(values[1:100])  # [20, 30]: the stop is clipped to the list end.
print(values[2:1])    # []: the default positive step cannot go backward.
print(values[3:3])    # []: equal start and stop select no elements.
print(values[-2:])   # [20, 30]: negative bounds count from the end.
```

Code 1.6: Clarifying the length of a slice at boundaries.

Runnable example: `ex07_slice_boundaries.py`

```
[20, 30]
[20, 30]
[]
[]
[20, 30]
```

Slicing beyond the end is therefore different from indexing beyond the end. A slice can return the available portion or an empty list. Direct indexing requests one actual element and raises an error when that element does not exist. A forgiving slice is convenient, but it can also conceal insufficient data: a program expecting five measurements should still check that five were supplied.

1.6. Open-ended slices, steps, and reverse order

Lecture-slide alignment: Week 5 source PDF, p. 16. PDF page numbers include title and divider pages.

In `x[:b]`, the omitted start means the beginning for the default positive step. In `x[a:]`, the omitted stop means the end. The third field in `x[start:stop:step]` sets how indices advance. A step of 2 selects every second position. A step of −1 moves backward; the omitted bounds in `x[::-1]` select the entire list in reverse order.

```
# Store sampling times in seconds.
time_s = [0, 1, 2, 3, 4, 5, 6]
first_three = time_s[:3]       # Select indices 0, 1, and 2.
from_two = time_s[2:]          # Select index 2 through the end.
every_second = time_s[::2]    # Select indices 0, 2, 4, and 6.
backward = time_s[::-1]       # Construct a list in reverse order.
# Print each result on its own line for inspection.
print(first_three)
print(from_two)
print(every_second)
print(backward)
```

Code 1.7: Open-ended and stepped slices (source PDF p. 16).

Runnable example: `ex06_slice_patterns.py`

```
[0, 1, 2]
[2, 3, 4, 5, 6]
[0, 2, 4, 6]
[6, 5, 4, 3, 2, 1, 0]
```

The operation `time_s[::2]` selects the first, third, fifth, and seventh stored samples in this example. It does not average neighbouring values. The result may represent coarser sampling, but only because the original times are known. Similarly, reversing a time list does not simulate time reversal; it merely changes the order in which the existing values appear.

With a negative step, the stopping bound is still excluded. The indices for `data[4:1:-1]` are 4, 3, and 2. The direction changes; the excluded-stop convention does not. A zero step is invalid because it would not advance through positions.

1.7. Prediction activity: index or slice?

Lecture-slide alignment: Week 5 source PDF, p. 17. PDF page numbers include title and divider pages.

Before running the following listing, write each result and its type. For every slice, list the selected indices rather than guessing from the values.

```python
# Work with the exact list in the slide activity.
data = [5, 10, 15, 20, 25, 30]
A = data[2]          # Select one element at index 2.
B = data[-2]         # Select one element, second from the end.
C = data[1:4]        # Select indices 1, 2, and 3.
D = data[:3]         # Select indices 0, 1, and 2.
E = data[::2]        # Select indices 0, 2, and 4.
F = data[4:1:-1]     # Select indices 4, 3, and 2; exclude index 1.
# Compare each output with a hand prediction.
print(A, B)
print(C)
print(D)
print(E)
print(F)
```

Code 1.8: Predict index and slice results A–F (source PDF p. 17).

Runnable example: `ex08_prediction.py`

Name	Selected indices	Result	Type
A	2	`15`	`int`
B	−2 (forward 4)	`25`	`int`
C	1, 2, 3	`[10, 15, 20]`	`list`
D	0, 1, 2	`[5, 10, 15]`	`list`
E	0, 2, 4	`[5, 15, 25]`	`list`
F	4, 3, 2	`[25, 20, 15]`	`list`

For this numerical dataset, A and B are scalar integers because they select individual integer elements. The other results are new lists. An element in a different dataset could itself be a list or tuple; indexing returns the selected element, whatever its type.

1.8. Length and membership answer different questions

Lecture-slide alignment: Week 5 source PDF, p. 18. PDF page numbers include title and divider pages.

The function `len()` answers how many elements a collection contains. The operator `in` asks whether an equal value occurs among the elements. Neither tells you the engineering significance of that value without the surrounding names and units.

```python
# Store available nominal diameters, all in millimetres.
standard_diameters_mm = [6, 8, 10, 12, 16]
number_available = len(standard_diameters_mm)  # Count available sizes.
has_10_mm = 10 in standard_diameters_mm  # Search for the value 10.
has_14_mm = 14 in standard_diameters_mm  # Search for the value 14.
# Membership tests produce Boolean results, not positions.
print(number_available, has_10_mm, has_14_mm)
```

Code 1.9: Length and membership in standard sizes (source PDF p. 18).

Runnable example: `ex09_membership.py`

```
5 True False
```

The test `10 in standard_diameters_mm` searches for the diameter value 10, not index 10. It returns `True`, not the location of the match. A size occurring more than once would still produce only one Boolean answer. To count occurrences or locate positions using the skills studied here, use an appropriate loop.

Stop, predict, and try it

Explain why a list of five standard sizes can contain the value 16 even though index 16 is invalid. Then predict `len([])` and `10 in []`. The answers are 0 and `False`: an empty collection has no elements to count or match.

1.9. Mean, minimum, maximum, and range

Lecture-slide alignment: Week 5 source PDF, p. 19. PDF page numbers include title and divider pages.

For a nonempty collection of n compatible numerical values, the arithmetic mean and range are

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i, \qquad R = x_{\max} - x_{\min}.$$

The subscript convention in mathematics starts at 1 here; the corresponding Python list positions start at 0. This is a notation difference, not a different dataset.

```python
# All entries are temperature measurements in deg C.
readings_c = [22.1, 22.5, 23.0, 23.4, 22.9]
# Divide the total by the number of measurements to find the mean.
average_c = sum(readings_c) / len(readings_c)
minimum_c = min(readings_c)   # Find the smallest measured value.
maximum_c = max(readings_c)   # Find the largest measured value.
range_c = maximum_c - minimum_c  # Calculate the temperature span.
# Formatting changes the display, not the underlying values.
print(f"Average = {average_c:.2f} deg C")
print(f"Range = {range_c:.2f} deg C")
```

Code 1.10: Summarize a nonempty temperature list (source PDF p. 19).

Runnable example: `ex10_summaries.py`

```
Average = 22.78 deg C
Range = 1.30 deg C
```

The five temperatures sum to 113.9 °C as an arithmetic intermediate. Dividing by 5 gives 22.78 °C. The smallest and largest readings are 22.1 and 23.4 °C, giving a range of 1.3 °C. Here `range_c` is a descriptive variable; it is not Python's `range()` function.

1.9.1. What the summary does not say

A mean discards order. The same five values in a different sequence have the same arithmetic mean, even though their peak time and trends can differ. A small range does not prove calibration, and a large range does not by itself prove a sensor fault. Those interpretations require the experimental context. The slides therefore warn against treating a summary as a complete description of measurement quality.

1.9.2. Empty measurements are not measured zeroes

With an empty list, `sum([])` is 0 and `len([])` is 0. The mean expression divides by zero. `min([])` and `max([])` also cannot choose an element and raise `ValueError` unless a separate policy is supplied. A clear beginner approach is to check the length first.

```
# This deliberately empty measurement list contains no evidence.
readings_c = []
# Check the count before division, min(), or max().
if len(readings_c) == 0:
    print("No temperature readings: summary unavailable.")
else:
    # This calculation is meaningful only for a nonempty list.
    average_c = sum(readings_c) / len(readings_c)
    print(f"Average = {average_c:.2f} deg C")
```

Code 1.11: An explicit empty-data guard for the slide summary.

Runnable example: `ex11_empty_guard.py`

This guard is an explicit expansion of the slide's empty-list warning. It reports that a summary is unavailable rather than inventing a value of zero. A one-element list is different: its mean, minimum, and maximum all equal the one reading, and its range is zero.

1.10. Loop directly over values

Lecture-slide alignment: Week 5 source PDF, p. 20. PDF page numbers include title and divider pages.

When an algorithm needs each measurement but not its position, direct iteration is the clearest pattern. On each pass, the loop variable refers to the next element in the existing order.

```
# All pressures and the limit use the same unit: kPa.
pressures_kpa = [95, 102, 108, 97]
warning_limit_kpa = 100
# Each pass binds pressure_kpa to one value, not an index.
for pressure_kpa in pressures_kpa:
    # Equality is not a warning under this strict greater-than rule.
    if pressure_kpa > warning_limit_kpa:
        print(f"Warning: {pressure_kpa} kPa")
```

Code 1.12: Loop directly over values (source PDF p. 20).

Runnable example: `ex12_value_loop.py`

```
Warning: 102 kPa
Warning: 108 kPa
```

The first and final pressures do not produce warnings because they are at or below 100 kPa. The variable `pressure_kpa` is singular because one element is considered at a time; `pressures_kpa` is the collection. Names like these make a loop easier to read aloud.

The warning boundary is strict: a value of exactly 100 kPa does not qualify. That is a property of `>`, not a universal rule about pressure. The code implements the threshold stated for this example.

1.11. Loop over indices when position matters

Lecture-slide alignment: Week 5 source PDF, p. 21. PDF page numbers include title and divider pages.

A shared index can connect two parallel lists. In the following example, `time_s[i]` and `temperature_c[i]` describe the same sampling event.

```
# Matching positions describe the same measurement event.
time_s = [0, 1, 2, 3]
temperature_c = [20.0, 21.8, 23.5, 25.1]
# Precondition: the lists have equal lengths and matching order.
for i in range(len(time_s)):
    # Use i for both lists so each time stays with its temperature.
    print(f"t = {time_s[i]} s, "
          f"T = {temperature_c[i]:.1f} deg C")
```

Code 1.13: Align two lists using the same index (source PDF p. 21).

Runnable example: `ex13_index_loop.py`

```
t = 0 s, T = 20.0 deg C
t = 1 s, T = 21.8 deg C
t = 2 s, T = 23.5 deg C
t = 3 s, T = 25.1 deg C
```

`len(time_s)` is 4, so `range(len(time_s))` produces 0, 1, 2, and 3. On each pass, the same index retrieves one time and its paired temperature. The adjacent f-strings inside `print()` form one output message.

Engineering check

Equal lengths are necessary for a complete one-to-one pairing, but are not sufficient evidence of correct alignment. Two four-element lists can still be in different orders. If the second list is shorter, indexing may fail; if it is longer, extra values may never be visited. Check both the lengths and the meaning of the positions before processing.

An index is useful when the position itself matters, when several aligned sequences must be accessed, or when an element will be deliberately replaced. Avoid adding indexing machinery when only the values are needed.

1.12. Use enumerate() for index and value together

Lecture-slide alignment: Week 5 source PDF, pp. 22–23. PDF page numbers include title and divider pages.

`enumerate(readings_c)` supplies pairs containing the current counter and element. In the default form, its counter starts at zero. The loop header unpacks each pair into two names, making both pieces of information directly available.

```python
# Temperatures are stored in acquisition order, in deg C.
readings_c = [22.1, 22.5, 23.0]
# Unpack the index and value supplied on each pass.
for index, reading_c in enumerate(readings_c):
    # The default index sequence starts at 0.
    print(f"Sample {index}: {reading_c:.1f} deg C")
```

Code 1.14: Use enumerate() to obtain position and value (source PDF p. 22).

Runnable example: `ex14_enumerate.py`

```
Sample 0: 22.1 deg C
Sample 1: 22.5 deg C
Sample 2: 23.0 deg C
```

The printed sample number matches the list index in this default version. A label chosen to start at 1 would be a human-readable sample number rather than the same zero-based index; do not interchange those meanings without saying so.

Pattern	Information exposed	Typical use
`for value in values`	Each element	Summary, threshold test, counter.
`for i in range(len(values))`	Each valid forward index	Aligned lists, replacement by position.
`for i, value in enumerate(values)`	Both index and element	Labelled report, event location.

Design principle

Choose the simplest loop that reveals the information needed by the algorithm. More syntax is not automatically more precise. The best pattern makes the relationship between data and computation easy to inspect.

1.13. Trace the first threshold crossing

Lecture-slide alignment: Week 5 source PDF, p. 24. PDF page numbers include title and divider pages.

```python
# Store an ordered sequence of temperatures in deg C.
temperature_c = [20.0, 24.5, 29.2, 35.1, 41.0]
limit_c = 30.0
first_crossing = None  # None means that no qualifying index is known yet.
# Inspect index-value pairs in their existing order.
for i, value_c in enumerate(temperature_c):
    # The boundary is inclusive: exactly 30.0 also qualifies.
    if value_c >= limit_c:
        first_crossing = i  # Store the position, not the temperature.
        break               # Stop after the first qualifying reading.
# A result of None would mean that no stored sample met the rule.
print(first_crossing)
```

Code 1.15: Find the first reading at or above a threshold (source PDF p. 24).

Runnable example: `ex15_first_crossing.py`

Index	Temperature (°C)	$T \geq 30$	Stored result	Action
0	20.0	False	`None`	Continue
1	24.5	False	`None`	Continue
2	29.2	False	`None`	Continue
3	35.1	True	3	Store index and break

The loop visits indices 0 through 3. Index 4 is never visited because `break` exits the loop immediately after the first match. The printed result is 3, meaning the fourth stored sample, not a temperature of 3 °C or an elapsed time of 3 seconds.

The initial value `None` means "no qualifying index has been found." It would remain `None` for an empty list or a list whose values are all below the threshold. Index 0 is a valid successful result; it must not be mistaken for failure. Testing `first_crossing is None` explicitly distinguishes these cases.

What "first crossing" means in this slide example

The code locates the first *stored sample at or above the limit*. It does not interpolate the exact physical crossing time and does not require an earlier below-limit sample. If the first reading already exceeds the limit, the answer is index 0. Sampling times would be needed to turn the index into a time.

1.14. Preserve quantity, units, and acquisition order

Lecture-slide alignment: Week 5 source PDF, p. 25. PDF page numbers include title and divider pages.

The bare list `[22.1, 22.5, 23.0]` does not identify a physical quantity. It could be temperature, voltage, or pressure. A name such as `temperature_readings_c`, a comment stating acquisition order, and a report heading including degrees Celsius provide complementary information.

A program should also distinguish sample index from physical coordinate. In the opening warm-up, the acquisition interval is explicitly one second. That information does not automatically apply to every later dataset. Similarly, the largest value in a list is not necessarily the latest value. `max(readings)` and `readings[-1]` answer different questions.

1.15. Debugging clinic: three different misconceptions

Lecture-slide alignment: Week 5 source PDF, p. 26. PDF page numbers include title and divider pages.

The slide shows three fragments using `readings = [2.1, 2.3, 2.2]`. Study them separately; if the first failing statement is run in a complete script, execution stops before later fragments are reached.

Case	Faulty expression or pattern	Diagnosis
A	`print(readings[3])`	Index 3 does not exist. The intended last forward index is 2.
B	`sum(readings) / 4`	Runs, but divides by an incorrect count. It gives 1.65 rather than 2.20.
C	`range(len(readings) + 1)` followed by indexing	Generates one extra index. Values at 0, 1, and 2 print before index 3 fails.

```python
# Three dimensionless values from the debugging activity.
readings = [2.1, 2.3, 2.2]
print(readings[-1])  # Repair A: retrieve the last existing element.
# Repair B: use the actual count, not an unrelated hard-coded 4.
average = sum(readings) / len(readings)
print(f"Average = {average:.2f}")
# Repair C: range ends before len(readings); do not add 1.
for i in range(len(readings)):
    print(readings[i])  # Each index is now valid.
```

Code 1.16: Repairs for the three collection mistakes (source PDF p. 26).

Runnable example: `ex16_collection_repairs.py`

Case B illustrates why an exception is not the only sign of a bug. A plausible number may be wrong because the program uses the wrong denominator. Derive the expected result independently: $(2.1 + 2.3 + 2.2)/3 = 2.2$.

1.16. Lecture 13 exit ticket and summary

Lecture-slide alignment: Week 5 source PDF, p. 27. PDF page numbers include title and divider pages.

There are five elements in the exit-ticket list, so its last valid forward index is 4. The expression `values[-2]` selects 12, and `values[1:4]` gives `[7, 9, 12]`. The worked answer below uses `enumerate()` to provide both index and value.

```python
# Use the list specified in the exit ticket.
values = [4, 7, 9, 12, 15]
print(len(values) - 1)  # Last valid forward index: 4.
print(values[-2])       # Second-to-last value: 12.
print(values[1:4])      # Values at indices 1, 2, and 3.
# Provide both index and value for a labelled report.
for index, value in enumerate(values):
    print(index, value)
```

Code 1.17: Worked Lecture 13 exit ticket (source PDF p. 27).

Runnable example: `ex17_exit13.py`

Changing the order can change engineering interpretation because positions often encode time, spatial location, or a defined sequence of design alternatives. A set of unchanged numbers is not the same as an unchanged experiment record.

2 Lecture 14: List Mutability and List Operations

Lecture-slide alignment: Week 5 source PDF, pp. 28–52. PDF page numbers include title and divider pages.

Learning outcomes

By the end of this chapter, you should be able to replace list elements; build and filter lists; distinguish `append()` from `extend()`; use `insert()`, `remove()`, and `pop()`; distinguish in-place changes from returned values; avoid deletion during direct iteration; explain aliasing and shallow copying; and recognize when a function changes its caller's data.

2.1. Warm-up: trace every change

Lecture-slide alignment: Week 5 source PDF, p. 30. PDF page numbers include title and divider pages.

The first exercise combines several operations. Do not try to predict only the final printed list. Write the complete state after each statement, its length, and the value returned by each method call.

```
# Begin with three values, in this order.
values = [10, 20, 30]
values[1] = 25        # Replace the second element; length stays 3.
values.append(40)    # Add one element at the end; length becomes 4.
values.insert(0, 5) # Insert before index 0; length becomes 5.
removed = values.pop()  # Remove and return the final element, 40.
print(values)        # Show the remaining four elements.
print(removed)       # Show the value returned by pop().
```

Code 2.1: Trace replacement, append, insert, and pop (source PDF p. 30).

Runnable example: `ex18_mutation_trace.py`

Statement or call	List afterwards	Length	Call result
Create list	`[10, 20, 30]`	3	—
`values[1] = 25`	`[10, 25, 30]`	3	—
`values.append(40)`	`[10, 25, 30, 40]`	4	`None`
`values.insert(0, 5)`	`[5, 10, 25, 30, 40]`	5	`None`
`values.pop()`	`[5, 10, 25, 30]`	4	`40`

An assignment statement such as `values[1] = 25` is not a method call that supplies a return value. The methods have separate behaviour: `append()` and `insert()` change the list and return `None`, whereas `pop()` changes the list and returns the removed element. This distinction is central to the lecture.

2.2. Mutable means the existing object can change

Lecture-slide alignment: Week 5 source PDF, pp. 31–32. PDF page numbers include title and divider pages.

A **mutable** object can be changed after it is created. Replacing an element of a list does not require binding the list variable to an entirely new list. The existing list object can retain its identity while one of its element positions refers to a different value.



Figure 2.1.: Mutation changes a list's contents. The highlighted position is replaced, not inserted. Based on source PDF p. 31.

Consider the confirmed temperature-entry error in the slide example:

```
# Temperatures in deg C; the original record contains a confirmed typo.
readings_c = [22.1, 22.5, 230.0, 23.4]
# Source scenario: independent evidence confirms that index 2 is 23.0.
readings_c[2] = 23.0
# This operation overwrites the old entry in the same list.
print(readings_c)
```

Code 2.2: Replace a confirmed data-entry mistake (source PDF p. 32).

Runnable example: `ex19_confirmed_correction.py`

```
[22.1, 22.5, 23.0, 23.4]
```

Python simply follows the replacement instruction. It does not know that 230.0 is a transcription error or that 23.0 is the correct replacement. The *source scenario* explicitly describes a confirmed data-entry mistake. In an actual dataset, an unusual value could instead be a real event, a unit mismatch, or a sensor fault. Keep the evidence that justifies the correction.

Replacing one element leaves the list length unchanged. Inserting an element, by contrast, increases its length and shifts later positions. Mixing up those intentions can break alignment with other measurements.

2.3. Calibration by changing each indexed element

Lecture-slide alignment: Week 5 source PDF, p. 33. PDF page numbers include title and divider pages.

Suppose the documented calibration indicates that the sensor reads 0.3 °C too high. The correction is

$$T_{\text{corrected},i} = T_{\text{raw},i} - 0.3\,^{\circ}\text{C}.$$

The sign is part of the calibration model. A positive bias in the raw reading is subtracted, not added.

```python
# Measured temperatures in deg C.
readings_c = [22.1, 22.5, 23.0, 23.4]
offset_c = 0.3  # The stated calibration says the sensor reads 0.3 high.
# Visit every existing index without changing the list length.
for i in range(len(readings_c)):
    # Replace the raw value with the corrected value at the same index.
    readings_c[i] = readings_c[i] - offset_c
# Raw values have now been overwritten in this list.
print(readings_c)
```

Code 2.3: Apply the slide calibration in place (source PDF p. 33).

Runnable example: `ex20_offset_in_place.py`

The loop uses indices because the left side of its assignment identifies a place in the list: `readings_c[i]`. The right side calculates a new temperature from the old value. After the statement completes, that place holds the corrected value. Every original entry is overwritten.

For the stated inputs, the intended decimal values are 21.8, 22.2, 22.7, and 23.1 °C. Printing a whole list can expose small binary floating-point representation effects; formatted individual values may display the expected one-decimal representation. Those display differences do not justify rounding every intermediate result.

2.3.1. Assigning to a loop variable is not the same operation

A direct loop binds a name such as `reading_c` to each element. Rebinding that name to `reading_c - 0.3` does not assign a new value into the original list. To replace a list element, assign through an index; to preserve the original, build a new processed list. A worked comparison appears in the cumulative practice chapter.

Common mistake

Re-running an in-place calibration example without restoring the raw readings applies the offset a second time. Always begin a comparison from known data, and document whether a function mutates its input or returns a separate result.

2.4. Safer calibration: build a processed list

Lecture-slide alignment: Week 5 source PDF, p. 34. PDF page numbers include title and divider pages.

An alternative preserves the measurements and creates a second list for the correction. It separates two different facts: what was measured, and what the stated calibration procedure produces.

```python
# Retain the original measurements as a separate record (deg C).
raw_readings_c = [22.1, 22.5, 23.0, 23.4]
offset_c = 0.3  # Subtract this documented positive bias (deg C).
corrected_readings_c = []  # Begin an empty processed list.
# Read each original value without replacing it.
for raw_c in raw_readings_c:
    # Add one corrected value to the new list in the original order.
    corrected_readings_c.append(raw_c - offset_c)
print(raw_readings_c)        # The original data remain available.
print(corrected_readings_c)  # The processed data form another list.
```

Code 2.4: Preserve the raw list while calibrating (source PDF p. 34).

Runnable example: `ex21_offset_new_list.py`

The empty brackets allocate a new output list. Each call to `append()` adds one corrected value at the end. Because the loop reads the original list in order, the corrected values preserve that same order. The output has one element for every input element.

A useful check is to compare the lengths and then inspect a known pair. The first raw reading remains 22.1 °C, while its corrected counterpart is approximately 21.8 °C. If the processed list is shorter, the program may have accidentally filtered rather than only calibrated.

Engineering check

Preservation is a design decision, not a magic property of a variable name. Calling a variable `raw_readings_c` does not make it read-only. Avoid operations that overwrite it, keep an explicit working list, and compare the retained record with the processed result.

2.5. Building and filtering with append()

Lecture-slide alignment: Week 5 source PDF, p. 35. PDF page numbers include title and divider pages.

Filtering constructs a collection containing the elements that satisfy a stated rule. The slide example retains pressures no greater than 105 kPa.

```python
# Store all measurements, including those above the stated limit.
all_pressures_kpa = [95, 108, 99, 115, 102]
limit_kpa = 105
safe_pressures_kpa = []  # Slide name: "safe" means only <= this limit.
# Inspect every candidate without deleting from the original list.
for pressure_kpa in all_pressures_kpa:
    if pressure_kpa <= limit_kpa:  # Equality is included.
        safe_pressures_kpa.append(pressure_kpa)  # Keep this value.
# Retained values preserve their original relative order.
print(safe_pressures_kpa)
```

Code 2.5: Keep values that satisfy the stated pressure rule (source PDF p. 35).

Runnable example: `ex22_filter_pressures.py`

```
[95, 99, 102]
```

The pattern has three parts: start with an empty output list, inspect each candidate, and append only candidates whose condition is true. Values 108 and 115 are not retained, while 95, 99, and

102 keep their relative order.

The name `safe_pressures_kpa` is retained from the slide. Here it means only "pressures satisfying the stated comparison". It is not a complete engineering safety judgment. In particular, filtering out warning values does not make the original process safe and should not hide those values from a report.

2.5.1. Filtering changes the meaning of later indices

The first retained element may still be the first original element, but later retained positions generally no longer identify the original sample numbers. In this example, retained index 1 holds 99 kPa, which came from original index 2. Keep the original indices alongside values when event locations matter. Lists of tuples later provide a natural way to do that.

2.6. Append one object; extend by its elements

Lecture-slide alignment: Week 5 source PDF, p. 36. PDF page numbers include title and divider pages.

These operations answer different representation questions. `append(object)` adds exactly one new outer element. `extend(iterable)` adds the elements supplied by an iterable as separate outer elements.

```
# Start two separate lists with the same initial contents.
a = [1, 2, 3]
b = [1, 2, 3]
# Add the complete object [4, 5] as one new element.
a.append([4, 5])
# Add 4 and 5 as two separate elements.
b.extend([4, 5])
print(a)        # [1, 2, 3, [4, 5]]: length 4.
print(b)        # [1, 2, 3, 4, 5]: length 5.
print(len(a), len(b))  # Count outer elements, not nested numbers.
```

Code 2.6: Append one object or extend by several elements (source PDF p. 36).

Runnable example: `ex23_append_extend.py`

```
[1, 2, 3, [4, 5]]
[1, 2, 3, 4, 5]
4 5
```

The fourth element of `a` is itself the list `[4, 5]`. Thus `len(a)` is 4, and `a[3]` is a list. In `b`, 4 and 5 occupy separate positions, so the length is 5. The brackets visible inside `a` indicate a nested structure, not merely decorative grouping.

A sensor message may contain several fields that belong to one record. Appending the complete record preserves that grouping. A batch of separate measurements may instead need extending. Decide whether the incoming object is one record or a sequence of separate new elements before selecting the method.

Both methods return `None`. Do not write `a = a.append(value)`: that rebinds the name to `None` after mutating the old list. The correct list-building pattern is a standalone method call on the existing list.

2.7. Insert at a chosen position

Lecture-slide alignment: Week 5 source PDF, p. 37. PDF page numbers include title and divider pages.

```
# Times in seconds; the sample at 2 s has been recovered.
time_s = [0, 1, 3, 4]
# First 2 = insertion index; second 2 = new time value in seconds.
time_s.insert(2, 2)
# Existing elements at index 2 and beyond shift one position right.
print(time_s)
```

Code 2.7: Insert a recovered sampling time (source PDF p. 37).

Runnable example: `ex24_insert.py`

```
[0, 1, 2, 3, 4]
```

The signature `insert(index, value)` places the new element before the position identified by the index in the usual in-range case. In the example, the first argument 2 is a list position and the second argument 2 is a time in seconds. They happen to be numerically equal, but their roles are different.

The old value 3 moves from index 2 to index 3, and the old value 4 moves from index 3 to index 4. If a parallel temperature list exists, the recovered temperature must be inserted at the matching position as well. Adding only the time creates a broken record structure. A list of paired records reduces this particular risk because time and value can be moved together.

2.8. Remove by value or remove by position

Lecture-slide alignment: Week 5 source PDF, pp. 38–39. PDF page numbers include title and divider pages.

`remove(value)` searches for the first matching value, removes that element, and returns `None`. `pop(index)` removes the element at a specified index and returns it. With no index argument, `pop()` removes the last element.

```
# -999 is the declared error code in this instructional example.
readings = [10, -999, 12, -999]
# Remove only the first entry equal to -999; return value is None.
result = readings.remove(-999)
print(readings, result)
# Reset to the original contents to compare the second operation.
readings = [10, -999, 12, -999]
removed = readings.pop(1)  # Remove index 1 and return that element.
print(readings, removed)  # One later error code still remains.
```

Code 2.8: Compare removing by value and by position (source PDF p. 38).

Runnable example: `ex25_remove_pop.py`

```
[10, 12, -999] None
[10, 12, -999] -999
```

Both examples leave `[10, 12, -999]` for these particular inputs. The identical remaining list does not mean the operations are interchangeable. One identifies the target by value; the other

identifies it by position. Only `pop()` returns the removed value for later use.

The code `-999` is an instructional error marker in these examples, not a universal meaning of that number. A real sensor protocol must define its own missing-data and error conventions. Repeated marker values also require care: one call to `remove(-999)` removes only the first match.

```
# This dataset contains no declared error code.
readings = [10, 12, 14]
error_code = -999
# Check that a matching value exists before using remove().
if error_code in readings:
    readings.remove(error_code)  # Remove the first match, if present.
else:
    print("No error code found")  # Report absence rather than failing.
```

Code 2.9: Test membership before removing a value (source PDF p. 39).

Runnable example: `ex26_remove_guard.py`

If `remove()` cannot find the requested value, it raises `ValueError`. Membership testing can prevent that expected situation. `pop()` instead requires a valid index and raises `IndexError` when that index is unavailable, including when trying to pop from an empty list. The absence of an error marker should be reported honestly, but it does not prove that every other reading is valid.

2.9. Why deleting during iteration can skip values

Lecture-slide alignment: Week 5 source PDF, p. 40. PDF page numbers include title and divider pages.

The following code is syntactically valid but is a poor filtering algorithm. It is included as a *deliberate logic demonstration*, not as a recommended method.

```
# This code runs but does not reliably remove every 2.
values = [1, 2, 2, 3, 2, 4]
# DELIBERATE LOGIC ERROR: removal shifts positions during iteration.
for value in values:
    if value == 2:
        values.remove(value)  # Remove the first matching value.
# One 2 remains because the loop position advances after deletion.
print(values)
```

Code 2.10: Deliberate logic error: delete while iterating (source PDF p. 40).

Runnable example: `ex27_unsafe_deletion.py`

```
[1, 3, 2, 4]
```

Deleting an element shifts later elements left. Meanwhile, the loop advances its internal position to the next index. A value that shifts into an already passed position may never be inspected. There is an additional subtlety in this exact example: `remove(value)` removes the *first* matching value, which may not be the element currently being visited.

Visited index	Value	List after this pass	What happened
0	1	`[1, 2, 2, 3, 2, 4]`	No removal.
1	2	`[1, 2, 3, 2, 4]`	First 2 removed; the next 2 shifts into index 1.
2	3	`[1, 2, 3, 2, 4]`	The shifted 2 was skipped.
3	2	`[1, 3, 2, 4]`	`remove(2)` removes the first remaining 2 at index 1.

The iterator's next position is 4, but the shortened list now has length 4, so the loop finishes. One 2 remains. Tracing the exact states explains the result more reliably than saying only that "something was skipped".

Common mistake

For the direct-iteration patterns taught here, do not change the source list's length while traversing it. A correct method for replacing values at existing indices is not automatically a correct method for inserting or deleting values.

2.10. Safe filtering into a new list

Lecture-slide alignment: Week 5 source PDF, p. 41. PDF page numbers include title and divider pages.

```
# Preserve this original sequence.
values = [1, 2, 2, 3, 2, 4]
clean_values = []  # The result initially contains no elements.
# Visit every original value exactly once.
for value in values:
    if value != 2:  # Retain only values that are not the excluded value.
        clean_values.append(value)
print(clean_values)  # [1, 3, 4].
print(values)        # The original list remains unchanged.
```

Code 2.11: Filter into a new list without shifting the source (source PDF p. 41).

Runnable example: `ex28_safe_filter.py`

```
[1, 3, 4]
[1, 2, 2, 3, 2, 4]
```

The source list is never shortened, so the loop inspects all six original values. The result contains exactly the retained values in their original relative order. Keeping the source also makes it possible to explain how many entries were excluded and why.

This pattern behaves sensibly in boundary cases. An empty source produces an empty result. If all elements satisfy the rule, all are retained. If none satisfy it, the result is empty, and any later mean or RMS calculation must handle that situation rather than divide by zero.

2.11. Concatenation and repetition are not vector arithmetic

Lecture-slide alignment: Week 5 source PDF, p. 42. PDF page numbers include title and divider pages.

```python
# Two separate runs; their numerical unit must be documented in use.
run_1 = [1.2, 1.4, 1.5]
run_2 = [1.3, 1.5, 1.6]
combined = run_1 + run_2  # Concatenate, rather than add element by element.
zeros = [0] * 5           # Repeat the one-element list five times.
print(combined)           # Run 1 appears before run 2.
print(zeros)              # [0, 0, 0, 0, 0].
print([10, 20] * 2)       # Repetition gives [10, 20, 10, 20].
```

Code 2.12: Concatenate runs and repeat a sequence (source PDF p. 42).

Runnable example: `ex29_concatenate_repeat.py`

```
[1.2, 1.4, 1.5, 1.3, 1.5, 1.6]
[0, 0, 0, 0, 0]
[10, 20, 10, 20]
```

For lists, `+` joins sequences. It does not add corresponding numerical elements. Multiplying a list by an integer repeats the sequence. Thus `[10, 20] * 2` gives four elements, not the two doubled numbers `[20, 40]`.

The zero initialization `[0] * 5` is appropriate here because the elements are immutable integers. Repeating a mutable nested object would repeat references to that object rather than independently cloning it; that is another reason to keep the aliasing model in mind.

Combining two experimental runs also has a data-management meaning. The joined list no longer shows where one run ended unless that boundary is retained elsewhere. Do not concatenate measurements with incompatible units, different sensors, or separate time origins and then interpret them as one uninterrupted series without documenting the transformation.

2.12. Two questions for every list operation

Lecture-slide alignment: Week 5 source PDF, p. 43. PDF page numbers include title and divider pages.

The slides distinguish *mutation* from the *returned result*. These are independent questions: a method may change the object, return something, do both, or only provide a new result.

Operation	Original outer list	Returned result
`values.append(x)`	Changes	`None`
`values.extend(other)`	Changes	`None`
`values.insert(i, x)`	Changes	`None`
`values.remove(x)`	Changes	`None`
`values.pop(i)`	Changes	Removed element
`values.sort()`	Changes	`None`
`values.reverse()`	Changes	`None`
`sorted(values)`	Unchanged	New sorted list
`values + other`	Unchanged	New combined list
`values.copy()`	Unchanged	New shallow outer copy
`values[:]`	Unchanged	New shallow outer copy

“Changes” describes an operation that acts on the existing list; the visible order may happen to

stay the same, for example when sorting a list that is already sorted. For new outer lists, nested mutable elements can still be shared. That distinction is developed below.

2.13. Sorting: change the original or keep a ranked copy

Lecture-slide alignment: Week 5 source PDF, pp. 44–45. PDF page numbers include title and divider pages.

```python
# Load values in newtons, initially in measurement order.
loads_n = [800, 500, 1200]
loads_n.sort()  # Sort the existing list in ascending order.
print(loads_n)
# Restore the original ordering for a separate comparison.
loads_n = [800, 500, 1200]
ranked_n = sorted(loads_n)  # Return a new ascending list.
print(loads_n)   # The original measurement order is preserved.
print(ranked_n)  # The ranked order is stored separately.
```

Code 2.13: Sort in place or create a sorted list (source PDF p. 44).

Runnable example: `ex30_sorting.py`

```
[500, 800, 1200]
[800, 500, 1200]
[500, 800, 1200]
```

The first operation changes `loads_n` itself. The second uses `sorted()` to leave the input order intact and bind a new list to `ranked_n`. Ascending numerical order is the default for these lists of numbers.

Sorting can answer a ranking question, but it discards chronological order if applied directly to a time series. Keeping a separately ranked list lets you ask the ranking question without overwriting the original sequence. Do not sort one member of a pair of parallel lists independently of the other.

2.13.1. The None surprise

```python
# Work with the slide's load list (N).
loads_n = [800, 500, 1200]
# INTENTIONAL TRAP: sorting changes loads_n but returns None.
ranked_n = loads_n.sort()
print(loads_n)  # The list itself is now sorted.
print(ranked_n) # None, not the sorted list.
```

Code 2.14: The return value of sort() is None (source PDF p. 45).

Runnable example: `ex31_sort_none.py`

```
[500, 800, 1200]
None
```

The return value of `sort()` is not the sorted list. The assignment binds `ranked_n` to `None`, while the existing `loads_n` object has already been sorted. This is the same conceptual distinction introduced in Week 4: a function or method may perform an action without returning the

numerical or collection result you expected.

Use `loads_n.sort()` alone when mutation is intended, or `ranked_n = sorted(loads_n)` when a new result is needed. Neither choice is automatically best; the need to preserve the original data determines the design.

2.14. Reversal: mutate or preserve

Lecture-slide alignment: Week 5 source PDF, p. 46. PDF page numbers include title and divider pages.

```
# First demonstrate mutation of an existing list.
values = [1, 2, 3, 4]
values.reverse()  # Reverse the same list; no new result is returned.
print(values)
# Now begin again with the original order.
values = [1, 2, 3, 4]
backward = values[::-1]  # A slice creates a separate reversed list.
print(values)    # Original order remains [1, 2, 3, 4].
print(backward)  # Reversed order is [4, 3, 2, 1].
```

Code 2.15: Reverse in place or obtain a reversed list (source PDF p. 46).

Runnable example: `ex32_reversing.py`

`reverse()` changes the existing list and returns `None`. The slice `[::-1]` supplies a new outer list in the opposite order. Reversal does not sort numerical values: the reverse of `[3, 1, 2]` is `[2, 1, 3]`, not `[3, 2, 1]`.

State why reverse order is needed. Reading newest samples first in a report is different from declaring that the experiment occurred in reverse chronological order. The same caution applies to time-position pairs and other aligned records.

2.15. Aliasing: two names, one list

Lecture-slide alignment: Week 5 source PDF, pp. 47–48. PDF page numbers include title and divider pages.

Assignment binds a name to an object. It does not automatically copy that object. After `processed = raw`, both names identify the same list.

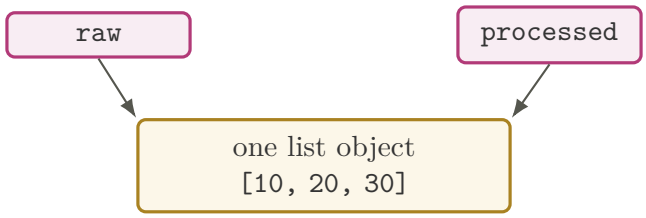


Figure 2.2.: Assignment creates a second name for the same list. Redrawn from the aliasing diagram on source PDF p. 47.

```python
# Create one list object and bind it to raw.
raw = [10, 20, 30]
processed = raw  # Bind another name to that same object.
processed[0] = 999  # Mutate the shared object through processed.
print(raw)        # The change is visible through both names.
print(processed)
```

Code 2.16: Assignment creates an alias, not a copy (source PDF p. 48).

Runnable example: `ex33_alias.py`

```
[999, 20, 30]
[999, 20, 30]
```

Changing an element through `processed` is visible through `raw` because there is only one list object. This is **aliasing**. It is not Python copying a later change back into the original: no separate original and processed lists ever existed.

Rebinding one name to a new list is different. If you subsequently write `processed = [0]`, the name `processed` refers to a different object, while `raw` still refers to the previously shared list. Distinguish *rebinding a variable* from *mutating an object* whenever you trace data changes.

2.16. A separate copy, and the meaning of shallow

Lecture-slide alignment: Week 5 source PDF, p. 49; copying terminology also on pp. 43 and 79. PDF page numbers include title and divider pages.

```python
# The original list holds immutable integer elements.
raw = [10, 20, 30]
processed = raw.copy()  # raw[:] is another shallow-copy option.
processed[0] = 999  # Replace an element only in the copied outer list.
print(raw)        # [10, 20, 30]: original values are preserved.
print(processed)  # [999, 20, 30]: working values have changed.
```

Code 2.17: Create a separate outer list before replacement (source PDF p. 49).

Runnable example: `ex34_copy.py`

```
[10, 20, 30]
[999, 20, 30]
```

`raw.copy()` and `raw[:]` create a new outer list. Replacing one of its elements does not replace the corresponding element of `raw`. For the flat list of integers in this slide, that is the desired independence.

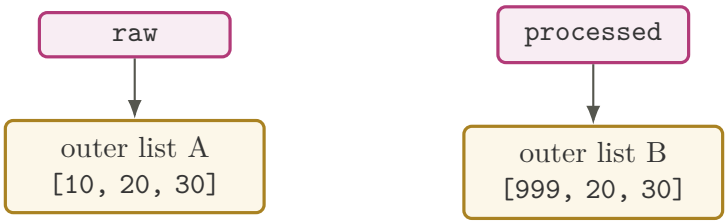


Figure 2.3.: After copying and replacing the first element of the copy, the two outer lists can hold different values. The diagram illustrates the flat numerical example on PDF p. 49.

Clarification of "independent copy" on PDF p. 49

The slides' reference tables call this a *shallow* copy. The outer list is new, but its elements are not recursively copied. Nested mutable elements may remain shared. Independence in the flat numerical example should not be interpreted as a guarantee that every object inside an arbitrarily nested dataset is duplicated.

```python
# Two measurement rows, each itself a mutable list.
raw_rows = [[0, 22.1], [1, 22.5]]
working_rows = raw_rows.copy()  # Copy only the outer list.
# The first row object is still shared by both outer lists.
working_rows[0][1] = 99.0
print(raw_rows)  # [[0, 99.0], [1, 22.5]]: a nested value changed.
# Compare with replacing the entire row in the working outer list.
working_rows[1] = [1, 24.0]
print(raw_rows)  # Its second row is still [1, 22.5].
```

Code 2.18: Technical clarification: a shallow copy can share a nested list.

Runnable example: `ex35_shallow_copy.py`

```
[[0, 99.0], [1, 22.5]]
[[0, 99.0], [1, 22.5]]
```

`working_rows[0][1]` first selects the first row list and then its second element. That row is shared, so changing its element is visible through both outer lists. Replacing the complete row `working_rows[1]` instead changes only a position in the copied outer list. This small explanatory example is included to make shallow copying precise, not to introduce a general nested-data processing topic.

2.17. Functions can mutate the caller's list

Lecture-slide alignment: Week 5 source PDF, p. 50. PDF page numbers include title and divider pages.

A parameter name is local to a function call, but the object it refers to may be shared with the caller. Therefore local scope does not guarantee that the caller's list is protected from mutation.

```python
def apply_offset_in_place(readings, offset):
    """Subtract offset from each reading, mutating the supplied list.

    All readings and offset must have the same unit.
    The function returns None; the caller's list is changed.
    """
    # Visit each index without adding or removing elements.
    for i in range(len(readings)):
        readings[i] = readings[i] - offset  # Replace the stored value.

# Raw temperature data in deg C.
raw_c = [22.1, 22.5, 23.0]
apply_offset_in_place(raw_c, 0.3)  # This call overwrites the raw values.
print(raw_c)  # The side effect is visible to the caller.
```

Code 2.19: A function can mutate the caller's list (source PDF p. 50).

Runnable example: `ex36_mutating_function.py`

During the call, `readings` and `raw_c` refer to the same list object. Indexed assignments change that shared object. The function has no explicit `return` statement, so it returns `None`; the changed data are visible because of the side effect, not because a new list was returned.

The name `apply_offset_in_place` is useful because it advertises mutation. Its docstring should state that the input changes, what units are expected, and which calibration is applied. A caller should not have to discover these facts through an accidental loss of raw data.

2.17.1. A worked alternative: return a new list

```python
def corrected_copy(readings, offset):
    """Return readings minus offset without mutating the input list.

    Use one consistent unit for all values and the offset.
    An empty input returns an empty list.
    """
    corrected = []  # Allocate a fresh output list on each call.
    for reading in readings:
        corrected.append(reading - offset)  # Preserve the input order.
    return corrected  # Return only after all values have been processed.

# Keep the original temperature readings (deg C).
raw_c = [22.1, 22.5, 23.0]
processed_c = corrected_copy(raw_c, 0.3)
print(raw_c)        # Original data.
print(processed_c)  # Corrected data.
```

Code 2.20: Worked alternative: return a new calibrated list.

Runnable example: `ex37_nonmutating_function.py`

This function explicitly constructs and returns a fresh list. The local result list is created on every call, each output element is appended in order, and `return` occurs after the loop. An empty input produces an empty output rather than a fabricated reading. The caller retains the raw series and receives a separately named processed series.

Function contract

For a function that accepts a collection, document more than input and output types. State whether the function mutates the supplied object, changes its length or order, creates a new result, or shares nested objects. Those behaviours are part of a usable interface.

2.18. A traceable data-processing workflow

Lecture-slide alignment: Week 5 source PDF, p. 51. PDF page numbers include title and divider pages.

The slide's workflow begins with a retained raw record, creates an explicit working representation, performs documented processing, compares and verifies, and reports the method, units, and exclusions.

Stage	Question to answer
Preserve raw data	What exactly was measured, and in which order?
Create a working list	Is this a real outer copy or only an alias?
Process deliberately	What calibration, replacement, or filtering rule was applied?
Compare and verify	Are retained lengths, values, units, and pairings consistent with that rule?
Report	Which data were changed or excluded, and why?

"Read-only record" in the slide is a handling policy. A normal Python list is not technically made immutable by that label. Likewise, a copied list cannot by itself explain why values were changed. Traceability needs both the original evidence and a documented transformation.

2.19. Lecture 14 exit ticket and summary

Lecture-slide alignment: Week 5 source PDF, p. 52. PDF page numbers include title and divider pages.

Append versus extend. Append adds one object as one outer element; extend adds the supplied iterable's elements separately. Both mutate the list and return `None`.

Remove versus pop. Remove targets the first matching value and returns `None`; pop targets a position and returns the removed element. Their failure conditions differ because one searches by value and the other indexes by position.

Sort versus sorted. Sort changes the existing list and returns `None`; sorted returns a new list. Preserve a separate original series whenever acquisition order matters.

Aliasing versus copying. Assignment creates another name for the same object. A shallow copy creates a new outer list but does not recursively duplicate nested objects.

For the laboratory calibration question, a defensible default is to retain the raw list and create a processed list. Keep the original values and order, the calibration offset and its sign, the unit, any verified replacement evidence, and the identities of excluded readings. In-place modification can be intentional, but the original must remain recoverable by a documented means when traceability is required.

3 Lecture 15: Tuples, Structured Records, and Laboratory 2

Lecture-slide alignment: Week 5 source PDF, pp. 53–77. PDF page numbers include title and divider pages.

Learning outcomes

By the end of this chapter, you should be able to create empty, singleton, and multi-element tuples; use familiar sequence operations; explain tuple immutability; pack and unpack values; return related results from a function; represent coordinates, bounds, and paired observations; process lists of tuples; choose a collection by intended use; and apply Week 5 skills to the sensor-data laboratory theme.

3.1. Warm-up: which structure should change?

Lecture-slide alignment: Week 5 source PDF, p. 55. PDF page numbers include title and divider pages.

Consider a temperature series that receives new measurements, the fixed coordinates of one sensor, and a growing collection of candidate beam widths. All three have an order, but they do not necessarily need the same kind of mutability.

A list is a natural working representation for incoming temperatures or changing design candidates. A tuple is a natural representation for a short fixed coordinate, such as (x, y) in metres. The choice expresses intent: which part of the program is allowed to grow, shrink, or replace an element?

Warm-up: a justified choice is more important than a label

Use a list for the growing measurement series, a tuple for the fixed sensor coordinate, and a list for the growing candidate widths. These are choices for the stated uses, not universal rules attached to those nouns. A finalized series might later be converted to a tuple, while an editable coordinate during setup might begin as a list.

3.2. A tuple is an immutable ordered sequence

Lecture-slide alignment: Week 5 source PDF, p. 56. PDF page numbers include title and divider pages.

A Python **tuple** is ordered like a list: positions have a stable meaning, indexing begins at zero, and a loop can visit elements in sequence. Its key difference is **immutability**: the tuple's existing element positions cannot be individually assigned new values.

```python
# A fixed (x, y) coordinate; both components are in metres.
sensor_position_m = (3.0, 4.0)
# Document positions as (lower limit, upper limit), in deg C.
operating_range_c = (15.0, 35.0)
print(sensor_position_m)        # Parentheses appear in the tuple display.
print(type(sensor_position_m))  # Confirm the tuple type.
```

Code 3.1: Fixed coordinates and operating bounds (source PDF p. 56).

Runnable example: `ex38_first_tuple.py`

```
(3.0, 4.0)
<class 'tuple'>
```

The coordinate convention is (x, y), and the range convention is (lower, upper). Neither convention is enforced by the tuple syntax. A tuple can contain values in any order that the programmer supplies, so comments and meaningful names still matter.

3.3. Empty, singleton, and multi-element tuples

Lecture-slide alignment: Week 5 source PDF, p. 57. PDF page numbers include title and divider pages.

```python
empty = ()           # An empty tuple with zero elements.
not_a_tuple = (5)    # Parentheses only group the integer expression.
singleton = (5,)     # The comma creates a one-element tuple.
pair = (5, 8)        # Two comma-separated values create a pair.
print(type(not_a_tuple))  # <class 'int'>.
print(type(singleton))    # <class 'tuple'>.
```

Code 3.2: Empty, singleton, and multi-element tuples (source PDF p. 57).

Runnable example: `ex39_singleton.py`

```
<class 'int'>
<class 'tuple'>
```

Parentheses can group an ordinary expression. Therefore `(5)` is just the integer 5. The trailing comma in `(5,)` establishes a one-element tuple. The empty tuple is the special notation `()`, while multiple comma-separated values form a multi-element tuple.

This is different from list syntax: `[5]` already clearly denotes a list because square brackets construct it. When debugging tuple code, inspect `type(object)` rather than relying only on visual parentheses.

Stop, predict, and try it

Predict the lengths of `()`, `(5,)`, and `(5, 8)`. They are 0, 1, and 2. Asking for the length of `(5)` is different: that object is an integer, not a sequence.

3.4. Familiar sequence operations transfer to tuples

Lecture-slide alignment: Week 5 source PDF, p. 58; shared interface on p. 69. PDF page numbers include title and divider pages.

```
# Ordered components are (x, y, z), all in metres.
point_m = (3.0, 4.0, 1.5)
x_m = point_m[0]    # Read the first coordinate.
z_m = point_m[-1]   # Read the final coordinate.
xy_m = point_m[:2]  # Select a tuple containing the first two coordinates.
print(x_m, z_m, xy_m)
```

Code 3.3: Index and slice a tuple (source PDF p. 58).

Runnable example: `ex40_tuple_access.py`

```
3.0 1.5 (3.0, 4.0)
```

`point_m[0]` reads one element and `point_m[-1]` reads the last. The slice `point_m[:2]` returns a tuple containing the first two elements. A list slice returns a list; a tuple slice returns a tuple. The same included-start and excluded-stop rules apply.

Length, membership, and iteration also work. The following expanded demonstration makes the shared operations in the slide reference table executable.

```
# Use a short fixed record of dimensionless example values.
values = (5, 8, 11)
print(len(values))  # Count three elements.
print(8 in values)  # Membership searches for a value.
# Concatenation requires another tuple and creates a new tuple.
combined = values + (14,)
repeated = values * 2  # Repetition is not elementwise multiplication.
for value in values:
    print(value)  # Ordered iteration also works for tuples.
print(combined)
print(repeated)
```

Code 3.4: Shared sequence operations, without mutating the tuple.

Runnable example: `ex50_tuple_shared_ops.py`

Concatenation joins two sequences of the same kind in these examples: list plus list or tuple plus tuple. Repetition duplicates the sequence of references, not each numerical magnitude. Neither operation modifies the original tuple. Tuple immutability does not prevent calculating a *new* tuple from existing values.

3.5. Immutability prevents item assignment

Lecture-slide alignment: Week 5 source PDF, pp. 59–60. PDF page numbers include title and divider pages.

```python
# The existing tuple is immutable.
point_m = (3.0, 4.0)
# DELIBERATE ERROR: a tuple does not support item replacement.
point_m[0] = 5.0
```

Code 3.5: Deliberate error: assigning into a tuple (source PDF p. 59).

Runnable example: `ex41_tuple_error.py`

```
TypeError: 'tuple' object does not support item assignment
```

The error arises because the existing tuple does not support replacement of its first element. This is an intentional constraint, not a malfunction. A validated coordinate or fixed pair of bounds should not be accidentally altered by the same indexed-assignment pattern used for lists.

A variable referring to a tuple is still reassignable. To deliberately use a different coordinate, bind the name to a new tuple. That changes what the name refers to; it does not change the old tuple. This is the same distinction between rebinding and object mutation used in the previous chapter.

3.5.1. Immutability communicates a limited constraint

Good tuple candidates in the slides include coordinates, lower/upper bounds, one calibration pair, fixed parameter groups, and related function results. Long records whose positions are hard to remember require especially careful documentation. Frequently corrected data or growing measurements often fit a list better.

Tuple immutability is not recursive protection

A tuple prevents replacing its element references. If one referenced element is itself mutable, that object's contents can still change. The slides' fixed coordinates and numerical pairs contain immutable numbers, so this distinction does not affect those particular examples. It matters when a tuple contains a list.

```python
# A tuple stores a fixed label and a reference to a mutable readings list.
record = ("T-01", [22.1, 22.5])
# The tuple's second element is not replaced; its list object is mutated.
record[1].append(23.0)
print(record)  # ('T-01', [22.1, 22.5, 23.0]).
```

Code 3.6: Technical clarification: tuple immutability is not recursive.

Runnable example: `ex51_tuple_mutable_element.py`

The second tuple element continues to refer to the same list. Calling that list's `append()` method changes the list, not the tuple's element reference. A tuple is therefore not automatically an immutable snapshot of all nested data or a guarantee of record authenticity.

3.6. Packing and unpacking

Lecture-slide alignment: Week 5 source PDF, p. 61. PDF page numbers include title and divider pages.

Packing combines several values into one tuple. **Unpacking** assigns elements of a sequence to several target names. In the simple unpacking pattern studied here, the number of names must

equal the number of elements.

```
x_m = 3.0  # Horizontal coordinate in metres.
y_m = 4.0  # Vertical coordinate in metres.
position_m = x_m, y_m  # Pack two values into one tuple.
print(position_m)
# Unpack exactly two elements into exactly two target names.
x_m, y_m = position_m
print(x_m, y_m)  # Print the recovered coordinate components.
```

Code 3.7: Pack a record, then unpack its values (source PDF p. 61).

Runnable example: `ex42_pack_unpack.py`

```
(3.0, 4.0)
3.0 4.0
```

The commas in `position_m = x_m, y_m` pack the values even without parentheses. The commas on the left in `x_m, y_m = position_m` mean that two names receive two elements in order. The first name receives element 0, and the second receives element 1.

Matching the number of elements prevents one class of error; matching their meaning prevents another. Unpacking `temperature, time = record` may run successfully even when the documented convention is `(time, temperature)`. Python matches positions, not the intended physical labels.

Function contract

A short positional record needs a convention: field count, field order, value types, units, and any required ranges. Ordinary unpacking requires the expected count, while meaningful names help a reviewer check the intended order.

3.7. Multiple assignment can swap two values

Lecture-slide alignment: Week 5 source PDF, p. 62. PDF page numbers include title and divider pages.

```
# These initial bounds are reversed.
low = 10.0
high = 2.0
# Change their order only when the lower bound exceeds the upper one.
if low > high:
    low, high = high, low  # Evaluate the old pair before rebinding names.
print(low, high)  # 2.0 10.0.
```

Code 3.8: Order two bounds with multiple assignment (source PDF p. 62).

Runnable example: `ex43_swap.py`

```
2.0 10.0
```

Python evaluates the right-hand side using the old values before rebinding the targets on the left. Thus `high, low` first supplies the pair `(2.0, 10.0)`, after which `low` becomes 2.0 and `high` becomes 10.0.

Two sequential assignments such as `low = high` followed by `high = low` would not perform

the same swap: the first assignment loses the old value of `low` from that name. The multiple-assignment pattern makes the simultaneous intent clear.

The slide connects this operation to ordered bounds in Week 3 bisection. Swapping reversed numerical bounds can establish `low <= high`, but it does not prove that a residual is defined, continuous, or sign-changing on the interval. The mathematical assumptions of the method remain separate responsibilities.

3.8. Functions return a tuple of related results

Lecture-slide alignment: Week 5 source PDF, p. 63. PDF page numbers include title and divider pages.

Week 4 introduced returning several related values. Tuples now explain the mechanism: a statement such as `return average, minimum, maximum` returns one tuple containing three elements. The caller may store that tuple or unpack its elements.

```python
def summarize(values):
    """Return (mean, minimum, maximum) for a nonempty numeric sequence.

    Every value must use the same physical unit.
    The input is read, not mutated. Empty input is not supported.
    """
    average = sum(values) / len(values)  # Compute the arithmetic mean.
    # Commas pack three results into one tuple in this fixed order.
    return average, min(values), max(values)

# Measurements in deg C.
readings = [22.1, 22.5, 23.0, 23.4]
avg, low, high = summarize(readings)  # Match the returned element order.
print(f"Mean = {avg:.2f}, min = {low:.1f}, max = {high:.1f} deg C")
```

Code 3.9: Return and unpack a summary tuple (source PDF p. 63).

Runnable example: `ex44_summary_tuple.py`

```
Mean = 22.75, min = 22.1, max = 23.4 deg C
```

The function's contract specifies a nonempty numeric sequence in one unit. It reads the input without modifying it. For the four temperatures shown, the mean is 22.75 °C, the minimum is 22.1 °C, and the maximum is 23.4 °C.

The return order is part of the interface. A caller assigning `low, avg, high` would create names with the wrong meanings even though three values fill three targets successfully. The function also does not automatically handle an empty list: the stated precondition must be respected or an explicit failure policy added.

3.9. Document the meaning of each tuple position

Lecture-slide alignment: Week 5 source PDF, p. 64. PDF page numbers include title and divider pages.

The slide's record is `measurement = (2.4, 36.8, 0.05)`. Its diagram gives the three positions the meanings time, temperature, and uncertainty.

Position	Meaning	Value	Unit
0	Time	2.4	s
1	Temperature	36.8	°C
2	Reported uncertainty	0.05	°C

```python
# Record convention: (time in s, temperature in deg C, uncertainty in deg C).
measurement = (2.4, 36.8, 0.05)
# Meaningful names make the positional convention visible.
time_s, temperature_c, uncertainty_c = measurement
print(f"Time = {time_s:.1f} s")
print(f"Temperature = {temperature_c:.2f} deg C")
print(f"Reported uncertainty = {uncertainty_c:.2f} deg C")
```

Code 3.10: Document the positions of a measurement tuple (source PDF p. 64).

Runnable example: `ex45_record_meaning.py`

Unpacking exposes the meaning that would otherwise remain hidden behind numerical indices. The tuple itself does not attach field labels, enforce units, or explain how the uncertainty was obtained. The slide does not identify the uncertainty as a standard deviation or confidence interval, so these notes do not assume either interpretation. In an actual report, document that definition before using the value in an uncertainty calculation.

3.10. A list of tuples stores paired data

Lecture-slide alignment: Week 5 source PDF, p. 65. PDF page numbers include title and divider pages.

```python
# Each tuple has the convention (input voltage, output voltage), in V.
calibration_data = [
    (0.0, 0.02),
    (1.0, 1.01),
    (2.0, 2.03),
    (3.0, 3.02)
]
# The outer list stores four records; each record has two elements.
print(len(calibration_data))
print(calibration_data[0])  # Read the complete first pair.
```

Code 3.11: A list of fixed calibration pairs (source PDF p. 65).

Runnable example: `ex46_calibration_records.py`

The outer list contains four elements. Each element is a two-value tuple, with the convention `(input_voltage, output_voltage)` in volts. There are therefore two levels of structure: the collection of observations, and the fields within one observation.

Adding another observation can append another tuple to the outer list. Reordering the list moves whole pairs rather than separating their fields. The outer list is still mutable, so a complete record can be replaced deliberately even though an existing tuple does not permit item assignment. The design communicates intent; it does not remove the need to document changes.

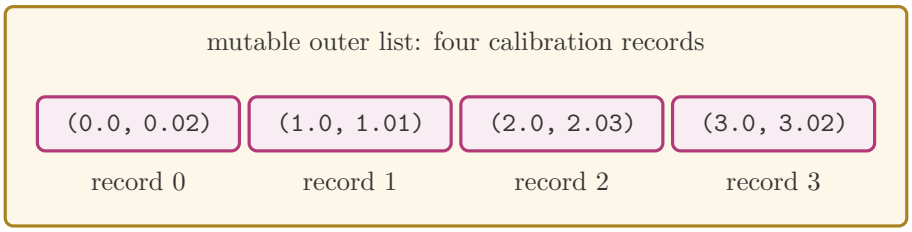


Figure 3.1.: The list may grow while each numerical calibration tuple groups one fixed input–output pair. This expands the two-level intent on source PDF p. 65.

3.11. Unpack records directly in a loop

Lecture-slide alignment: Week 5 source PDF, p. 66. PDF page numbers include title and divider pages.

```
# Use the three records from the slide's loop example; voltages are in V.
calibration_data = [(0.0, 0.02), (1.0, 1.01), (2.0, 2.03)]
# Each pass unpacks one two-element tuple into its meaningful names.
for input_v, output_v in calibration_data:
    error_v = output_v - input_v  # Signed output-minus-input deviation.
    # The + sign in +.2f prints an explicit sign for positive errors.
    print(f"Input = {input_v:.1f} V, "
          f"error = {error_v:+.2f} V")
```

Code 3.12: Unpack every calibration pair in the loop header (source PDF p. 66).

Runnable example: `ex47_calibration_loop.py`

```
Input = 0.0 V, error = +0.02 V
Input = 1.0 V, error = +0.01 V
Input = 2.0 V, error = +0.03 V
```

Each pass takes one pair, binds its input voltage to `input_v`, and binds its output voltage to `output_v`. The calculation is the signed difference

$$e_V = V_{\text{output}} - V_{\text{input}}.$$

For the three displayed observations, the differences are approximately +0.02, +0.01, and +0.03 V. The formatting specification `+.2f` includes an explicit plus sign for positive values and displays two decimal places.

The short loop uses the first three records shown in the preceding four-record slide. These are two related demonstrations, not conflicting dataset sizes. The loop precondition is that each record supplies exactly two fields in the expected order and unit.

The difference measures deviation from the identity relation in this example. It does not by itself establish a complete calibration model, measurement uncertainty, or a universal acceptance criterion. The stored pairs provide evidence; an engineering interpretation requires the stated measurement purpose.

3.12. Parallel lists versus paired records

Lecture-slide alignment: Week 5 source PDF, p. 67. PDF page numbers include title and divider pages.

The same three observations can be represented as `time_s = [0, 1, 2]` with `position_m =`

`[0, 2, 5]`, or as `[(0, 0), (1, 2), (2, 5)]` with the convention `(time_s, position_m)`.

Parallel lists can be convenient for separate processing, but they require a persistent alignment agreement. Removing the middle time without also removing its position leaves values paired with the wrong event. Equal lengths after an unrelated change do not guarantee correct alignment either.

A list of tuples makes each pair one element of the outer collection. Inserting, removing, or moving a complete record carries both values together. This addresses a structural risk, but does not prove that the pair was correct when it was first recorded.

Design principle

Store values together when preserving their relationship is more important than processing them independently. Document the within-record order and the order of records in the outer list.

3.13. Interval velocity from adjacent records

Lecture-slide alignment: Week 5 source PDF, p. 68. PDF page numbers include title and divider pages.

For two position observations separated by a nonzero time interval, the average velocity over that interval is

$$\bar{v} = \frac{x_1 - x_0}{t_1 - t_0}.$$

With positions in metres and times in seconds, the result is in metres per second. This is an interval average; it need not equal the instantaneous velocity at the interval endpoint.

```python
# Record convention: (time in seconds, position in metres).
position_data = [(0, 0.0), (1, 2.0), (2, 5.0), (3, 9.0)]
# Begin at index 1 so an earlier record exists at index i - 1.
for i in range(1, len(position_data)):
    t0, x0 = position_data[i - 1]  # Unpack the earlier measurement.
    t1, x1 = position_data[i]      # Unpack the later measurement.
    # Precondition: t1 > t0, with times and positions in consistent units.
    velocity = (x1 - x0) / (t1 - t0)  # Interval-average velocity (m/s).
    print(f"Interval ending at {t1} s: "
          f"v = {velocity:.1f} m/s")
```

Code 3.13: Calculate interval velocities from adjacent records (source PDF p. 68).

Runnable example: `ex48_paired_velocity.py`

```
Interval ending at 1 s: v = 2.0 m/s
Interval ending at 2 s: v = 3.0 m/s
Interval ending at 3 s: v = 4.0 m/s
```

The loop begins at index 1 so that an earlier record exists at index `i - 1`. Four observations produce three adjacent intervals. Beginning at index 0 would make `position_data[i - 1]` select the last record through negative indexing, creating a physically inappropriate first-to-last comparison rather than raising an obvious error.

Interval (s)	Δx (m)	Δt (s)	$\bar{v}$ (m/s)	Ending record index
0 to 1	2	1	2	1
1 to 2	3	1	3	2
2 to 3	4	1	4	3

For this time-series interpretation, times must be strictly increasing. A repeated time makes the denominator zero; a reversed time violates the intended chronology. A single record produces no interval, and an empty record list also produces no interval. Those outcomes mean that interval velocity is unavailable, not that it equals zero. The cumulative practice chapter supplies a clearly labelled guard for non-increasing times.

3.14. Compare the shared interface and the mutation constraint

Lecture-slide alignment: Week 5 source PDF, pp. 69–70. PDF page numbers include title and divider pages.

Operation or property	**List**	**Tuple**
Ordered elements	Yes	Yes
`len(x)`, `x[i]`, `x[a:b]`	Yes	Yes
Membership and ordered iteration	Yes	Yes
Concatenation with the same sequence type	New list	New tuple
Integer repetition	New list	New tuple
`x[i] = value`	Supported	Not supported
`append()`, `insert()`, `remove()`	Supported	Not tuple methods
`sort()` in place	Supported	Not a tuple method

The decisive difference for this week is mutability. Use a list when the collection will grow, shrink, be corrected, be sorted in place, or accumulate results. Use a tuple when a short stable record should resist individual element replacement or when related returned values belong together.

The type does not automatically specify the quantity, units, origin, or within-record field meanings. A tuple of two incorrect numbers is still incorrect. A carefully documented list may be more useful than an unexplained fixed tuple.

3.15. Convert only when the representation requirement changes

Lecture-slide alignment: Week 5 source PDF, p. 71. PDF page numbers include title and divider pages.

```python
# An editable coordinate list, in metres.
coordinates_list = [3.0, 4.0]
coordinates_tuple = tuple(coordinates_list)  # Create a fixed outer record.
# A fixed pair of example bounds (same quantity and unit).
fixed_bounds = (0.0, 100.0)
working_bounds = list(fixed_bounds)  # Create an editable outer list.
print(coordinates_tuple)
print(working_bounds)
```

Code 3.14: Convert representations deliberately (source PDF p. 71).

Runnable example: `ex49_conversions.py`

```
(3.0, 4.0)
[0.0, 100.0]
```

Converting the list to a tuple creates a tuple representation with the same element order. Converting the tuple to a list creates an editable outer list. In these cross-type examples the outer object is new; the conversion does not perform validation, numerical unit conversion, or a recursive copy of nested elements.

An editable coordinate might be accepted from a user and checked before being converted to a fixed tuple. That conversion communicates the next stage of use. It does not make an invalid coordinate valid. The validation and the representation change are separate steps.

3.16. Structure-selection activity: justify the intended use

Lecture-slide alignment: Week 5 source PDF, p. 72. PDF page numbers include title and divider pages.

Information	Reasonable choice	Justification and qualification
100 acceleration readings	List	Convenient for processing and correction; a finalized record could later be made fixed.
One 3-D coordinate	Tuple	A short validated (x, y, z) record should retain its element order.
Standard pipe diameters	List or tuple	Use a list if availability is edited; a tuple if a fixed catalogue is intended.
Fixed temperature bounds	Tuple	Lower and upper limits form a short stable ordered pair.
Candidate areas	List	Candidates may be added, removed, filtered, or ranked.
One time-temperature observation	Tuple	Both fields describe one observation and should move together.

The slide asks for reasoning, not only a type name. Ask whether the collection must grow or shrink, whether an element must be replaced, and whether immutability communicates a useful constraint. More than one answer may be defensible when the intended workflow has not been specified.

3.17. Laboratory 2: source dataset and proposed tasks

Lecture-slide alignment: Week 5 source PDF, pp. 73–74. PDF page numbers include title and divider pages.

The slide theme is to process an ordered acceleration dataset and produce a clear engineering report. The ten source readings, in acquisition order and in $\mathrm{m\,s^{-2}}$, are

$$[0.12,\ 0.18,\ 0.15,\ 0.22,\ 0.19,\ 0.30,\ 0.28,\ 0.25,\ 0.21,\ 0.17].$$

The proposed tasks are to store the list; inspect its first, last, and first five values; calculate mean, minimum, maximum, and range; compute RMS with a loop; count readings above a warning limit; preserve raw data and create a corrected copy; build sample-reading records; print labels and units; test hand-checkable cases; and explain the limitations of summaries.

Laboratory note

The slides do not give the final warning-limit value, calibration-offset value, sampling interval, deadline, or grading instructions. The following material is a *worked study example*, not a replacement for the final Moodle brief. In particular, no sampling interval is inferred from the ten readings, so a sample index must not be reported as a time in seconds.

A useful workflow is to inspect the original data before applying any transformation. Then calculate the unprocessed summaries, construct a separate processed list, and show enough information to relate each processed value to its original sample. A report should distinguish a warning condition from an invalid measurement: exceeding a limit does not automatically justify deleting a reading.

3.18. RMS: square, average, then take the square root

Lecture-slide alignment: Week 5 source PDF, p. 75. PDF page numbers include title and divider pages.

For $n > 0$ real measurements, the **root-mean-square** value is

$$x_{\mathrm{RMS}} = \sqrt{\frac{1}{n}\sum_{i=1}^{n} x_i^2}.$$

Read the operation order carefully. First square *each* value. Then average those squares. Finally take the square root. Squaring the mean or taking the square root of the ordinary sum computes something else.

If each acceleration is in $\mathrm{m\,s^{-2}}$, its square is in $\mathrm{m^2\,s^{-4}}$. The arithmetic average has the same squared unit. Taking the square root returns the result to $\mathrm{m\,s^{-2}}$. The count n is dimensionless.

The slide demonstrates the pattern using *four* readings, not the complete ten-reading dataset:

```python
# Acceleration readings in m/s^2; this list is nonempty.
readings = [0.12, 0.18, 0.15, 0.22]
sum_of_squares = 0.0  # Start the squared-value accumulator before the loop.
for value in readings:
    sum_of_squares += value**2  # Add one squared acceleration (m^2/s^4).
mean_square = sum_of_squares / len(readings)  # Average the squares.
rms = mean_square**0.5  # Take the square root; unit returns to m/s^2.
print(f"RMS = {rms:.3f} m/s^2")
```

Code 3.15: RMS accumulator for the four-reading slide example (source PDF p. 75).

Runnable example: `ex52_slide_rms.py`

```
RMS = 0.172 m/s^2
```

The manual calculation is

$$\begin{aligned}\sum x_i^2 &= 0.12^2 + 0.18^2 + 0.15^2 + 0.22^2 \\ &= 0.0144 + 0.0324 + 0.0225 + 0.0484 = 0.1177, \\ \frac{1}{4}\sum x_i^2 &= 0.029425, \\ x_{\mathrm{RMS}} &= \sqrt{0.029425} \approx 0.171537 \ \mathrm{m\,s^{-2}}.\end{aligned}$$

Thus the three-decimal printed result is $0.172\ \mathrm{m\,s^{-2}}$. An accumulator trace makes the role of each loop pass visible.

Pass	Reading	Squared reading	Running sum of squares
1	0.12	0.0144	0.0144
2	0.18	0.0324	0.0468
3	0.15	0.0225	0.0693
4	0.22	0.0484	0.1177

Common mistake

Initialize `sum_of_squares` once before the loop. Resetting it inside the loop loses the earlier contributions. Place the division and final square root after accumulation is complete. An empty collection requires an explicit policy because dividing by its length is not meaningful.

3.18.1. Interpreting RMS without overclaiming

The squaring makes RMS nonnegative. Positive and negative readings do not cancel in the sum of squares. For instance, the arithmetic mean of $[-3, 3]$ is zero, while its RMS is 3. This is a numerical explanation of the formula, not evidence that a particular acceleration signal is safe or correctly calibrated.

RMS also discards order: permuting the same readings leaves the sum of squares unchanged. It cannot tell you when a peak occurred, whether sampling was sufficiently frequent, or whether the instrument had a bias. It is not generally the standard deviation, because RMS is calculated from the values themselves rather than their deviations from their mean.

3.19. Worked study report for all ten readings

Lecture-slide alignment: Week 5 source PDF, pp. 73–75; the full report below is an added worked example. PDF page numbers include title and divider pages.

Illustrative settings, not supplied laboratory requirements

For this worked report only, use a warning rule *raw acceleration strictly greater than* 0.25 $\mathrm{m\,s^{-2}}$, and an assumed positive sensor offset of 0.02 $\mathrm{m\,s^{-2}}$ to subtract. These values are not specified in the slides. No readings are excluded, and the sample counter begins at zero. The report keeps raw summaries separate from corrected values.

3.19.1. Plan the variables before writing the program

The raw list is the retained record. `sample_count` identifies its length. `sum_of_squares` accumulates squared raw readings, while `warning_count` counts matching events. `warning_indices` preserves their original locations. A new list stores corrected values, and a list of tuples stores `(sample_index, raw_reading)` pairs.

Keeping these responsibilities distinct avoids two common mistakes: computing statistics on corrected values while labelling them raw, and discarding the sample identities when filtering or counting warnings. The warning rule is applied to the *raw* readings here; applying it to corrected data would answer a different question and must be stated explicitly.

The next three code panels are consecutive parts of one script. Preserve the indentation and run them together. The complete file is `ex59_lab_report.py`.

```
# Data and units are taken from the Week 5 slides (PDF p. 73).
raw_acceleration_m_s2 = [
    0.12, 0.18, 0.15, 0.22, 0.19,
    0.30, 0.28, 0.25, 0.21, 0.17
]
# ILLUSTRATIVE settings only: the slides do not specify these values.
warning_limit_m_s2 = 0.25  # Warn strictly above this limit on RAW data.
offset_m_s2 = 0.02         # Subtract this assumed positive sensor bias.

# Guard summaries against an empty dataset.
if len(raw_acceleration_m_s2) == 0:
    print("No readings: a numerical summary is unavailable.")
else:
    sample_count = len(raw_acceleration_m_s2)  # Number of readings.
    first = raw_acceleration_m_s2[0]    # First recorded acceleration.
    last = raw_acceleration_m_s2[-1]    # Latest recorded acceleration.
    first_five = raw_acceleration_m_s2[:5]  # A new five-element window.

    # Compute summaries of the ORIGINAL readings, all in m/s^2.
    mean_m_s2 = sum(raw_acceleration_m_s2) / sample_count
    minimum_m_s2 = min(raw_acceleration_m_s2)
    maximum_m_s2 = max(raw_acceleration_m_s2)
    range_m_s2 = maximum_m_s2 - minimum_m_s2
```

Code 3.16: Worked sensor report, part 1 of 3: data, guards, and raw summaries.

Runnable example: `ex59_lab_report.py`

```
24
25      # Initialize every accumulator and result list before the loop.
26      sum_of_squares = 0.0
27      warning_count = 0
28      warning_indices = []
29      corrected_acceleration_m_s2 = []
30      records = []
31
32      # Each pass preserves the original index and measurement value.
33      for index, acceleration in enumerate(raw_acceleration_m_s2):
34          sum_of_squares += acceleration**2  # Squared-value total.
35          if acceleration > warning_limit_m_s2:  # Strict boundary rule.
36              warning_count += 1
37              warning_indices.append(index)  # Keep event locations.
38          # Build a new calibrated list without overwriting raw data.
39          corrected = acceleration - offset_m_s2
40          corrected_acceleration_m_s2.append(corrected)
41          # A record is (zero-based sample index, raw acceleration in m/s^2).
42          records.append((index, acceleration))
43
44      # Finish RMS only after every squared value has been accumulated.
45      rms_m_s2 = (sum_of_squares / sample_count)**0.5
```

Code 3.17: Worked sensor report, part 2 of 3: initialize accumulators and process every sample.

Runnable example: `ex59_lab_report.py`

```
46
47      # Report the dataset, rules, units, and results explicitly.
48      print("ACCELERATION STUDY REPORT")
49      print(f"Number of readings = {sample_count}")
50      print(f"First = {first:.2f} m/s^2; last = {last:.2f} m/s^2")
51      print(f"First five (m/s^2) = {first_five}")
52      print(f"Mean = {mean_m_s2:.3f} m/s^2")
53      print(f"Minimum = {minimum_m_s2:.2f} m/s^2")
54      print(f"Maximum = {maximum_m_s2:.2f} m/s^2")
55      print(f"Range = {range_m_s2:.2f} m/s^2")
56      print(f"RMS = {rms_m_s2:.6f} m/s^2")
57      print(f"RAW warning rule: acceleration > {warning_limit_m_s2:.2f} m/s^2")
58      print(f"Warnings = {warning_count}; indices = {warning_indices}")
59      print(f"Illustrative subtracted offset = {offset_m_s2:.2f} m/s^2")
60      print("Index | Raw (m/s^2) | Corrected (m/s^2)")
61      # Matching indices link the original records to corrected values.
62      for index, raw_value in records:
63          corrected = corrected_acceleration_m_s2[index]
64          print(f"{index:5d} | {raw_value:12.2f} | {corrected:18.2f}")
65      print("Raw data preserved; no readings excluded.")
66      print("Sample index is not elapsed time; sampling times are unspecified.")
```

Code 3.18: Worked sensor report, part 3 of 3: produce the labelled report.

Runnable example: `ex59_lab_report.py`

3.19.2. How to read the integrated program

The first block stores the slide dataset and the two explicitly assumed settings. The empty-data condition prevents every summary that requires at least one reading. Inside the nonempty branch,

the first, last, and window accesses are well-defined for the supplied ten-element dataset.

The loop processes each original sample once. Squaring contributes to RMS; the strict comparison contributes to warning evidence; subtracting the offset constructs a new calibrated value; packing the index and raw reading creates one fixed record. None of those steps replaces an element in the raw list. The report loop then uses the retained index to look up the corrected value belonging to the same observation.

The long program is not intended for memorization. Trace one pass, then ask which names represent state before the loop, which change on each pass, and which represent results after the loop. This combines the Week 2 loop pattern, Week 4 decomposition habits, and Week 5 collection semantics.

3.19.3. Independent verification of the numerical results

For the complete ten-reading dataset,

$$\sum x_i = 2.07, \qquad \bar{x} = \frac{2.07}{10} = 0.207 \text{ m s}^{-2},$$

and

$$\sum x_i^2 = 0.4577, \qquad x_{\text{RMS}} = \sqrt{0.4577/10} \approx 0.213939244 \text{ m s}^{-2}.$$

The minimum is 0.12, the maximum is 0.30, and the range is 0.18 m s^{-2}. The first five readings are `[0.12, 0.18, 0.15, 0.22, 0.19]`; the last reading is 0.17 m s^{-2}.

Under the illustrative *strict* warning rule, only 0.30 and 0.28 exceed 0.25. They occur at zero-based indices 5 and 6, so the warning count is 2. The value 0.25 at index 7 does not count. Changing the comparison to `>=` would count three readings instead: that would be a changed specification, not a repair to the same rule.

```
ACCELERATION STUDY REPORT
Number of readings = 10
First = 0.12 m/s^2; last = 0.17 m/s^2
First five (m/s^2) = [0.12, 0.18, 0.15, 0.22, 0.19]
Mean = 0.207 m/s^2
Minimum = 0.12 m/s^2
Maximum = 0.30 m/s^2
Range = 0.18 m/s^2
RMS = 0.213939 m/s^2
RAW warning rule: acceleration > 0.25 m/s^2
Warnings = 2; indices = [5, 6]
Illustrative subtracted offset = 0.02 m/s^2
Index | Raw (m/s^2) | Corrected (m/s^2)
    0 |         0.12 |               0.10
    1 |         0.18 |               0.16
    2 |         0.15 |               0.13
    3 |         0.22 |               0.20
    4 |         0.19 |               0.17
    5 |         0.30 |               0.28
    6 |         0.28 |               0.26
    7 |         0.25 |               0.23
    8 |         0.21 |               0.19
    9 |         0.17 |               0.15
Raw data preserved; no readings excluded.
Sample index is not elapsed time; sampling times are unspecified.
```

The calibration arithmetic subtracts 0.02 from every raw reading. Since every value receives the same subtraction, the corrected arithmetic mean is $0.207 - 0.02 = 0.187\ \mathrm{m\,s^{-2}}$. The range remains 0.18 because subtracting the same offset from both extremes does not change their difference. RMS, however, is not generally shifted by the same subtraction: its square-and-average operation must be recalculated.

3.19.4. An honest engineering interpretation

For this supplied dataset, the raw mean is $0.207\ \mathrm{m\,s^{-2}}$ and the raw RMS is approximately $0.213939\ \mathrm{m\,s^{-2}}$. Under the example threshold, two of ten raw observations exceed the warning limit. All raw readings remain stored, and a separate processed list records the assumed offset correction.

These statements summarize the specified data and rule. They do not establish the physical origin of the acceleration, the adequacy of the warning threshold, the validity of the offset, the sampling rate, or instrument accuracy. Six decimal places in the numerical display make the calculation reproducible; they do not imply measurement accuracy to six decimal places.

3.20. Hand-checkable tests for the laboratory pattern

Lecture-slide alignment: Week 5 source PDF, pp. 74–75; worked tests added. PDF page numbers include title and divider pages.

Testing should include simple data for which the result is independent of the program. A useful RMS test uses `[0, 0, 0, 4]`. The squared sum is 16, the mean square is 4, and the RMS is 2. A constant list `[3, 3]` should return 3; a zero list should return zero; a signed pair `[-3, 3]` should also return 3.

The function below makes the test interface explicit. Its empty-input policy is an *added design choice*: return `None` to distinguish unavailable data from measured zero. The original short slide listing simply assumes nonempty data.

```python
def calculate_rms(values):
    """Return RMS for finite, moderate-sized real values in one unit.

    Return None for an empty sequence. Do not mutate the input.
    This introductory function does not validate element types.
    """
    if len(values) == 0:
        return None  # Distinguish no data from a measured zero.
    sum_of_squares = 0.0  # Initialize once per function call.
    for value in values:
        sum_of_squares += value**2  # Accumulate every squared value.
    return (sum_of_squares / len(values))**0.5  # Same unit as the input.

# Independent hand-checkable tests; units are assumed consistent.
assert calculate_rms([]) is None  # Empty data do not imply zero RMS.
assert calculate_rms([0, 0]) == 0
assert calculate_rms([3, 3]) == 3
assert calculate_rms([-3, 3]) == 3
# 0^2 + 0^2 + 0^2 + 4^2 = 16; divide by 4, then take sqrt: 2.
assert calculate_rms([0, 0, 0, 4]) == 2
print("Hand-checkable RMS tests passed.")
```

Code 3.19: A reusable RMS function with an explicit empty-input policy.

Runnable example: `ex60_rms_function.py`

The contract limits the routine to finite, moderate-sized real numbers in one unit. It is not a complete validator for arbitrary strings, missing values, infinities, or extremely large magnitudes. The code expresses the elementary formula taught in the slides.

```python
# Purpose-built test values below, at, and above the illustrative limit.
test_readings = [0.20, 0.25, 0.30]
limit = 0.25  # Same unit as every test reading.
warning_count = 0
for value in test_readings:
    if value > limit:  # Exactly 0.25 must not increment this counter.
        warning_count += 1
assert warning_count == 1  # Only 0.30 is strictly above the limit.
print("Strict-boundary test passed.")
```

Code 3.20: Boundary checks for a strictly-above warning rule.

Runnable example: `ex61_threshold_tests.py`

The warning test deliberately includes a value at the boundary. Other checks should confirm that calibration preserves list length and order, leaves the raw list unchanged, and returns an empty output for an empty input when using the nonmutating calibration function from Lecture 14.

Test	Input or setup	Expected behaviour
Empty data	`[]`	No mean; no minimum or maximum; explicit unavailable-data policy.
One measurement	`[0.2]`	Mean, minimum, maximum, and RMS equal 0.2; range is zero.
Constant data	`[3, 3]`	Mean and RMS are 3; range is zero.
Signed data	`[-3, 3]`	Mean is 0; RMS is 3.
Warning boundary	`[0.20, 0.25, 0.30]` with limit 0.25	Strictly-above count is 1.
Copy preservation	Copy then replace one outer element	Original flat list remains unchanged.
Record consistency	Every sample-reading tuple has two fields	Loop unpacking succeeds with two names.

3.21. Debugging clinic: tuple and record mistakes

Lecture-slide alignment: Week 5 source PDF, p. 76. PDF page numbers include title and divider pages.

The four slide fragments are separate exercises. Running them in one script without handling the first error would prevent the later cases from executing.

3.21.1. A: a grouped number is not a coordinate pair

```
# Parentheses alone do not make a tuple.
point = (3.0)
# DELIBERATE ERROR: a float cannot supply two elements for unpacking.
x, y = point
```

Code 3.21: Deliberate error A: grouped number is not a two-value record (source PDF p. 76).

Runnable example: `ex53_tuple_bug_a.py`

The value of `point` is a float, so it cannot be unpacked into two names. Adding only a trailing comma gives a singleton tuple, but it *still* cannot fill two names. A two-dimensional point requires two coordinate values, for example `(3.0, 4.0)`. The repair must match the intended record, not only remove the first error.

3.21.2. B: a tuple does not support element replacement

```
# A fixed tuple stores the lower and upper bounds.
bounds = (0.0, 100.0)
# DELIBERATE ERROR: the tuple does not permit item assignment.
bounds[1] = 120.0
```

Code 3.22: Deliberate error B: attempting to overwrite a tuple element (source PDF p. 76).

Runnable example: `ex54_tuple_bug_b.py`

The attempted assignment raises `TypeError`. Decide whether bounds should be editable. A list is appropriate when individual elements must be edited; alternatively, deliberately construct a new tuple and rebind `bounds`. Do not claim that rebinding changes the old tuple.

3.21.3. C: the records have inconsistent lengths

```
# The second record violates the intended two-field convention.
records = [(0, 20.0), (1, 21.5, "C")]
# DELIBERATE ERROR: the second pass has three values but only two names.
for time_s, temperature_c in records:
    print(time_s, temperature_c)  # The first record prints before failure.
```

Code 3.23: Deliberate error C: inconsistent record length (source PDF p. 76).

Runnable example: `ex55_tuple_bug_c.py`

The first two-field record prints. On the next pass, the three-field record cannot be unpacked into two names, so a `ValueError` occurs. A defensible repair is to retain the unit information and use a consistent three-field convention in every record. Simply discarding a unit field to make the code run could erase meaningful information.

3.21.4. D: the return count and target count differ

```
# DELIBERATE ERROR: three returned values cannot fill just two names.
average, low = (22.5, 21.0, 24.0)
```

Code 3.24: Deliberate error D: too few target names (source PDF p. 76).

Runnable example: `ex56_tuple_bug_d.py`

Three values are supplied but only two names are present. Use three meaningful names when all three summary values belong to the interface. The mismatch causes a `ValueError`; a wrong ordering with the *correct count* could instead run and silently give the names incorrect meanings.

3.21.5. A coherent set of repairs

```
# Repair A: a two-dimensional point needs two values, not just a comma.
point = (3.0, 4.0)
x, y = point  # Unpack exactly two coordinates.
# Repair B: bind the name to a new tuple instead of mutating the old one.
bounds = (0.0, 100.0)
bounds = (bounds[0], 120.0)  # Deliberately replace the whole record.
# Repair C: preserve the existing unit field and make every record consistent.
records = [(0, 20.0, "C"), (1, 21.5, "C")]
for time_s, temperature_c, unit in records:
    print(time_s, temperature_c, unit)  # All three meanings remain visible.
# Repair D: provide one name for each of the three summary values.
average, low, high = (22.5, 21.0, 24.0)
print(average, low, high)
```

Code 3.25: Repair record structure and unpacking deliberately (source PDF p. 76).

Runnable example: `ex57_tuple_repairs.py`

The repairs change the data convention only where that decision is explicit. They distinguish a two-coordinate point from a singleton, a new tuple from item mutation, consistent records from accidental field removal, and the number of returned values from the names used to receive them.

3.22. Concept map, exit ticket, and Lecture 15 summary

Lecture-slide alignment: Week 5 source PDF, p. 77. PDF page numbers include title and divider pages.

The concept map begins with an *ordered engineering collection*. You access positions through indexing and slicing, select mutability through the list/tuple choice, and process the collection using loops and functions. Units and meaning must remain explicit throughout these operations.

The exit ticket asks for one record containing time, voltage, and current, and an outer collection containing three such records. The values below are illustrative worked-answer values; the slide does not prescribe measurements.

```
# Added illustrative values; convention is (time in s, voltage in V, current in A).
electrical_records = [
    (0.0, 12.0, 0.50),
    (1.0, 12.0, 0.55),
    (2.0, 11.9, 0.60)
]
# The list can grow; each three-value tuple keeps one observation together.
for time_s, voltage_v, current_a in electrical_records:
    print(f"t = {time_s:.1f} s, "
          f"V = {voltage_v:.1f} V, I = {current_a:.2f} A")
```

Code 3.26: Worked Lecture 15 exit ticket: electrical records (source PDF p. 77).

Runnable example: `ex58_exit15.py`

Each tuple fixes the ordered fields of one observation. The outer list can grow as observations arrive. The first field is time in seconds, the second voltage in volts, and the third current in amperes. A different order would also be possible if declared and consistently used, but changing it silently would make the data ambiguous.

Section summary

Tuples express short, fixed ordered records; lists express collections that can grow or change. A list of tuples combines those intentions. Reliable processing preserves paired fields, checks assumptions such as nonempty data and valid time intervals, computes transparent summaries, and reports units, transformations, and limitations.

4 Week 5 Synthesis, Reference, and Worked Practice

Lecture-slide alignment: Week 5 source PDF, pp. 78–81; cumulative exercises are added study material. PDF page numbers include title and divider pages.

4.1. Read a collection program as an engineering argument

A collection program makes a chain of decisions before it prints a result. It chooses what one element represents, how elements are ordered, how a position is accessed, which objects may change, what information should be retained, and which summary supports the stated question.

The Week 5 closing slide emphasizes that a collection is not merely several values in brackets. Order, mutability, units, and structure express an engineering decision. The following reading method turns that message into a practical habit.

First identify the original dataset and the meaning of its order. Next find every operation that creates a new collection or introduces another name. Then mark which statements replace elements, change list length, or reorder data. Finally inspect the calculations, boundaries, and labels used to report results. Ask whether the final output can still be traced to the original measurements.

Engineering check

When a summary seems surprising, check the representation before changing the formula. The program may be averaging a filtered list, indexing a sorted list as though it were chronological, or working through an alias that already overwrote the raw data. A correct formula applied to the wrong collection still answers the wrong question.

4.2. Quick reference: sequence access and iteration

Lecture-slide alignment: Week 5 source PDF, p. 78. PDF page numbers include title and divider pages.

Expression or pattern	Meaning and condition
`x[0]`, `x[-1]`	First and last element; requires a nonempty collection.
`x[i]`	One element at a valid integer index.
`x[1:3]`	Elements at indices 1 and 2, if present.
`x[:3]`, `x[2:]`	A prefix or suffix, returned as the same sequence kind.
`x[::2]`	Every second position, beginning at 0.
`x[::-1]`	Reverse-order slice, not an in-place mutation.
`len(x)`	Number of outer elements.
`value in x`	Whether an equal value appears as an element.
`for value in x`	Ordered iteration over elements.
`for i in range(len(x))`	Ordered iteration over valid forward indices.
`for i, value in enumerate(x)`	Index and value together, with default indexing from zero.

For length $n > 0$, forward indices are 0 through $n - 1$ and negative indices are -1 through $-n$. Ordinary slices exclude their stopping bound. The simple slice-length rule $b - a$ requires the conditions explained in Section 1.5.

4.3. Quick reference: operations and tuple patterns

Lecture-slide alignment: Week 5 source PDF, p. 79. PDF page numbers include title and divider pages.

List operation	Essential distinction
`x[i] = value`	Replace an element at an existing position.
`x.append(value)`	Add one object as one element; return `None`.
`x.extend(other)`	Add the supplied elements separately; return `None`.
`x.insert(i, value)`	Insert at a chosen position; later positions shift.
`x.remove(value)`	Remove first matching value; return `None`.
`x.pop(i)`	Remove by index and return the removed value.
`x.sort()`	Reorder the existing list; return `None`.
`y = sorted(x)`	Obtain a new sorted list while preserving the input order.
`x.reverse()`	Reverse the existing list; return `None`.
`y = x.copy()`	Create a new shallow outer list; nested objects can remain shared.
`y = x`	Create another reference, not a copy.

Tuple pattern	Meaning
`point = (3.0, 4.0)`	Create a two-element tuple.
`one = (3.0,)`	Create a singleton; the comma is essential.
`x, y = point`	Unpack two elements into two names, in order.
`low, high = high, low`	Swap by evaluating the old pair before rebinding.
`return a, b`	Return a tuple of related values.
`for a, b in records`	Unpack every two-field record in a loop.
`list(point)`	Create an editable list representation.
`tuple(values)`	Obtain a tuple representation of the iterable's elements.

4.4. A working glossary

Collection
An object that groups related elements. This week concentrates on lists and tuples.

Sequence
A collection whose elements have a defined positional order.

Scalar
In these notes, one numerical or similar single-value quantity, contrasted with a sequence of measurements.

List
A mutable ordered sequence written with square brackets.

Tuple
An immutable ordered sequence, commonly written with parentheses and comma-separated elements.

Element
One object at a position in a collection. An element can itself be another collection.

Index
An integer position used to retrieve or deliberately replace an element when the collection permits replacement.

Zero-based indexing
A convention in which the first position is index 0.

Negative index
A position counted from the end, beginning with -1 for the last element.

Slice
A positional selection described by start, excluded stop, and optional step.

Half-open interval
An interval that includes its starting boundary and excludes its stopping boundary in the slice patterns studied here.

Membership
The question of whether a value occurs as an element; tested with `in` or `not in`.

Iterable
An object whose elements can be visited by a loop.

Accumulator
A variable holding a running total, such as the sum of squares in RMS.

Counter
A variable that counts events, such as readings above a threshold.

Mutation
A change to an existing object's contents or ordering.

Rebinding
Associating a variable name with a different object. Rebinding a name is not the same as mutating its previously referenced object.

In-place operation
An operation that acts on the existing object rather than replacing it with a returned new collection.

Return value
The object supplied by a function or method call for the caller to use. Many mutating list methods return `None`.

Aliasing
The situation in which multiple names or references refer to the same object.

Shallow copy
A new outer collection that initially contains references to the original elements; nested mutable objects are not recursively duplicated.

Side effect
A change beyond a function's returned value, such as mutation of a caller-supplied list.

Filtering
Selecting elements by a stated condition, often into a new list.

Concatenation
Joining sequences end to end without elementwise numerical addition.

Record
A group of fields describing one observation, coordinate, or other meaningful entity.

Packing and unpacking
Combining values into a tuple and assigning sequence elements to multiple target names.

Parallel lists
Separate sequences whose matching positions are intended to describe the same events.

Precondition
An assumption that must hold before an operation is used, such as nonempty data or strictly increasing times.

RMS
The square root of the arithmetic mean of squared values. It has the same physical unit as the input and is nonnegative for real inputs.

Traceability
The ability to reconstruct what was measured, how it was transformed, which entries were excluded, and why.

4.5. Cumulative practice questions

These questions are added reading-note exercises built from the slide concepts. They are not additional assessed requirements. Predict each result before running code, and explain the meaning as well as the syntax. Worked answers follow in Section 4.6.

P1. Positions and values. For `values = [8, 11, 14, 17, 20]`, state its length, last forward index, `values[-3]`, and `values[1:4]`. Which result is a list?

P2. Slice boundaries. For `values = [10, 20, 30]`, predict `values[1:100]` and `values[2:1]`. Explain why their lengths are not 99 and −1.

P3. First qualifying sample. A list contains `[30.0, 28.0, 31.0]`. What does the slide's first-crossing code return for an inclusive 30-degree limit? What would it return for `[20.0, 29.0]`? Explain why testing the result as a generic truth value would be risky.

P4. Operation trace. Begin with `[2, 4, 6]`. Append 8, insert 1 at index 0, pop index 2 into a variable, and reverse the remaining list. State the removed value and the final list. Which methods return `None`?

P5. One object versus several elements. Starting separately from `[1, 2]`, compare appending `[3, 4]` with extending by `[3, 4]`. Give the final lengths and final elements.

P6. Aliasing and rebinding. Trace Example 4.1 when you reach the worked solutions. Explain the final values of `a`, `b`, and `c` without using the phrase "Python copied the changes".

P7. Loop-variable trap. Why does assigning `reading_c = reading_c - 0.3` inside a direct value loop fail to calibrate the stored list? Give a repair that preserves raw data.

P8. Retain event identity. For the slide pressure series `[95, 108, 99, 115, 102]`, construct a list of `(original_index, pressure)` tuples for values strictly above 105 kPa.

P9. Singleton and field meaning. Explain why `(3.0)` is not a tuple, and why changing it to `(3.0,)` still cannot fill `x, y`. State a valid convention for one time-voltage-current record.

P10. Interval validity. For `[(0, 0.0), (1, 2.0), (1, 5.0)]`, which interval is valid for average velocity and which is not? Show how an explicit check can prevent an invalid division.

P11. Summary verification. Calculate the mean, minimum, maximum, range, and RMS of `[0, 0, 0, 4]`. Explain why RMS is not the same as the mean. What changes when the collection is empty?

P12. Audit a report. A report says: "The maximum is 0.30, the warning happened at 5 seconds, and calibration proves the system is safe." Using the ten-reading slide dataset, identify what is supported, what needs a unit or qualification, and what has not been supplied by the source.

4.6. Worked solutions and explanations

4.6.1. P1–P3: access, boundaries, and meaningful failure states

P1. The length is 5, the last forward index is 4, and `values[-3]` is 14. The slice `values[1:4]` is `[11, 14, 17]`, which is a list. Length and index are not the same kind of information even when both are integers.

P2. The first slice gives `[20, 30]` because the oversized stop is limited by the available data. The second gives `[]` because the default positive step cannot move from start 2 toward a smaller excluded stop 1. A collection length cannot be negative. The rule $b - a$ has conditions, rather than being a universal identity for every slice expression.

P3. The first list returns index 0 immediately because 30.0 satisfies the inclusive comparison. The second produces `None`. In a Boolean context, zero is false, but it is a perfectly valid index. Test `first_crossing is None` when distinguishing "not found" from a successful match at the first position.

4.6.2. P4–P5: trace mutation and distinguish element levels

P4. The states are:

Operation	State afterwards	Returned value
Start	`[2, 4, 6]`	—
Append 8	`[2, 4, 6, 8]`	`None`
Insert 1 at index 0	`[1, 2, 4, 6, 8]`	`None`
Pop index 2	`[1, 2, 6, 8]`	4
Reverse	`[8, 6, 2, 1]`	`None`

P5. Appending gives `[1, 2, [3, 4]]`, length 3, with final element `[3, 4]`. Extending gives `[1, 2, 3, 4]`, length 4, with final element 4. The nested brackets represent one inner list object; they cannot be ignored when counting outer elements.

4.6.3. P6: aliasing, copying, and rebinding

```python
# Create one list, one alias, and one shallow copy.
a = [4, 7, 9]
b = a
c = a[:]
b[1] = 70  # Mutates the shared a/b list.
c.append(12)  # Changes only the copied outer list.
b = [0]  # Rebinds b to a different list without changing a.
print(a)
print(b)
print(c)
```

Code 4.1: Practice trace: copying, mutation, and rebinding.

Runnable example: `ex63_alias_rebind.py`

```
[4, 70, 9]
[0]
[4, 7, 9, 12]
```

Initially `a` and `b` identify one list, while `c` identifies a shallow outer copy. The assignment through `b[1]` changes the shared list, so `a` sees 70. Appending to `c` changes only its outer copy. The final assignment `b = [0]` points `b` to a new list; it does not reverse the earlier mutation of `a`.

4.6.4. P7: why rebinding the loop variable does not replace list elements

```python
# Store the original temperatures in deg C.
readings_c = [22.1, 22.5, 23.0]
# This loop rebinds only reading_c; it does not replace any list element.
for reading_c in readings_c:
    reading_c = reading_c - 0.3
print(readings_c)  # Still the original three values.
# To replace entries deliberately, assign through each index instead.
processed_c = readings_c.copy()  # Keep the original list intact.
for i in range(len(processed_c)):
    processed_c[i] = processed_c[i] - 0.3
print(processed_c)
```

Code 4.2: Practice repair: assigning to a loop variable is not item assignment.

Runnable example: `ex62_rebinding_loop_variable.py`

The first loop only binds `reading_c` to each number and then rebinds it to a calculated number. No list position is assigned. The repaired loop works on a copy and assigns through `processed_c[i]`, explicitly replacing the corresponding element. The input list remains unchanged.

An equally valid repair is the new-list calibration pattern from Section 2.4. It reads each raw value and appends the corrected result, avoiding in-place replacement altogether.

4.6.5. P8: filter records while preserving original indices

```python
# The list is in acquisition order; all values use kPa.
pressures_kpa = [95, 108, 99, 115, 102]
limit_kpa = 105
warning_records = []
for index, pressure_kpa in enumerate(pressures_kpa):
    if pressure_kpa > limit_kpa:  # Select values that exceed the rule.
        # Preserve both original index and value in a tuple.
        warning_records.append((index, pressure_kpa))
print(warning_records)  # [(1, 108), (3, 115)].
```

Code 4.3: Practice solution: retain event locations when filtering.

Runnable example: `ex64_preserve_indices.py`

```
[(1, 108), (3, 115)]
```

The warning observations are 108 and 115 kPa at indices 1 and 3. Each tuple preserves both pieces of information. An index in this filtered list is not automatically the original sample index; the tuple explicitly retains that original identity.

4.6.6. P9: one field cannot supply two coordinate names

Parentheses around 3.0 group a float expression. A comma creates a singleton tuple, but that singleton still has only one element. A point with two coordinate fields needs two values, for example `(3.0, 4.0)`, or a different unpacking design with only one target if a single coordinate is intended.

A valid electrical record convention is `(time_s, voltage_v, current_a)`. Every record should follow that three-field order. Units and the meaning of the sampling time belong in comments and report headings; tuple syntax alone does not encode them.

4.6.7. P10: detect a non-increasing time interval before division

```python
# Deliberate repeated time illustrates an invalid interval.
position_data = [(0, 0.0), (1, 2.0), (1, 5.0)]
for i in range(1, len(position_data)):
    t0, x0 = position_data[i - 1]  # Earlier record: s and m.
    t1, x1 = position_data[i]      # Later record: s and m.
    if t1 <= t0:
        # Do not divide by zero or quietly accept reversed chronology.
        print(f"Invalid time interval at record index {i}")
    else:
        velocity = (x1 - x0) / (t1 - t0)  # Interval average in m/s.
        print(f"Interval {i}: {velocity:.1f} m/s")
```

Code 4.4: Practice solution: reject non-increasing adjacent times.

Runnable example: `ex65_guarded_velocity.py`

```
Interval 1: 2.0 m/s
Invalid time interval at record index 2
```

The first interval gives $(2-0)/(1-0) = 2$ m/s. The second has zero elapsed time, so the quotient

is not defined. Reporting an invalid interval preserves the evidence instead of silently changing a time or inventing a velocity. This is a worked extension of the precondition in the slide example, not a different formula.

4.6.8. P11: numerical summaries are different summaries

For `[0, 0, 0, 4]`, the mean is 1, minimum 0, maximum 4, and range 4. The RMS is

$$\sqrt{(0^2 + 0^2 + 0^2 + 4^2)/4} = \sqrt{4} = 2.$$

The mean uses the values themselves; RMS averages squared values before taking the square root. They therefore answer different questions and need not be equal. If the values carry a unit, both the mean and RMS carry that unit.

An empty collection supplies no measurements. The mean and RMS formulas cannot divide by a positive count, and a minimum or maximum cannot select an element. The program should apply a documented unavailable-data policy. Returning zero would confuse absence of measurement with an actual zero measurement.

4.6.9. P12: distinguish source evidence from an unsupported conclusion

The maximum raw reading is supported, but it should be reported as $0.30\ \mathrm{m\,s^{-2}}$. The source dataset identifies order, not sampling times. It does not establish that index 5 means 5 seconds. Under the illustrative raw warning rule used in this note, warnings occur at indices 5 and 6; the threshold itself is not supplied by the slides.

A calibration offset must be supported by measurement evidence. The worked offset of $0.02\ \mathrm{m\,s^{-2}}$ is explicitly illustrative. Even an independently valid calibration does not prove system safety; a safety conclusion needs a suitable physical model and justified acceptance criteria. The strongest report states the data, transformation, comparison rule, and limitations separately.

4.7. A readiness checklist for the next week

Before leaving Week 5, check that you can explain a program containing each of the following without relying on a memorized output: a negative index, a reverse slice, a boundary guard, a value loop, an indexed loop, `enumerate()`, a mutating method with a `None` return, an alias and a shallow copy, a function that changes a supplied list, a singleton tuple, a multi-result function, a list of paired records, and a loop-based RMS.

Then ask a more demanding question: can another reader determine what one element means, which unit it uses, which order is preserved, what changed, and why? That is the engineering level of understanding the slides are aiming for.

Key idea

Choose the representation deliberately. Access values through valid positions. Change only what you intend to change. Preserve enough evidence to reconstruct the original data and every reported result.

A Source Alignment and Technical Clarifications

A.1. Primary source and recommended reading

The primary teaching source is **Yanwei Wang, *Programming for Engineers: Week 5 — Tuples, Lists, and Ordered Engineering Data*, School of Engineering, Nazarbayev University, Fall 2026**, supplied as `ENG101_Week5_Complete_Slides.pdf`, 81 PDF pages. The note sequence follows its Lecture 13, Lecture 14, and Lecture 15 divisions. The explanations, trace tables, worked activity answers, and cumulative exercises are developed from that sequence.

The following recommended reading is listed on source PDF p. 80. It is retained here as recommended reading; the slide deck, rather than an independent synthesis of these books, defines the scope of this document.

1. Nazarbayev University, *ENG 101 — Programming for Engineers: Course Specification Report*, Fall 2026, Session 5.
2. Y. Wang, *Programming for Engineers with Python*, working draft, August 2026, Chapter 5.
3. J. V. Guttag, *Introduction to Computation and Programming Using Python*, 2nd edition, Chapter 5, Sections 5.1 and 5.3–5.5.

The preparation habit on the same slide is to predict indices and slices, trace each mutation, preserve raw data, and justify the list/tuple choice using engineering intent. These actions are the basis of the study method at the beginning of the notes.

A.2. What has been clarified or added

Presentation repairs.
Some slide headings contain the visible fragment "exttt" before a code name. These fragments have been omitted. Display-wrapped code has been placed on valid Python lines, with comments added. No code operation is inferred from a typesetting fragment.

Forward-index terminology: PDF p. 12.
The slides' "positive indices" include zero. The notes call these forward or nonnegative indices and retain the same indexing rule.

Slice length: PDF p. 15.
The statement that `x[a:b]` has length $b - a$ is qualified by step 1 and $0 \leq a \leq b \leq n$. Boundary demonstrations explain why oversized, reversed, or negative bounds need their usual Python interpretation.

Empty-data assumptions: PDF pp. 19, 63, and 75.
The source summary and RMS formulas require nonempty compatible numerical data. Added guards and the later RMS function state their own unavailable-data policy instead of implying that the short source listings already implement it.

Shallow copying: PDF pp. 43, 49, and 79.
The notes distinguish a new outer list from a recursive copy of nested data. A small nested-list

example is explicitly labelled as a technical clarification.

Function side effects: PDF p. 50.
The source phrase "returns nothing" means that the function returns `None`; its effect is mutation of the supplied list. A nonmutating alternative is marked as a worked alternative.

Tuple immutability: PDF pp. 59–60.
The notes distinguish protection against item assignment from rebinding a name or mutating a nested list. The latter behaviour is shown only in a labelled clarification.

Threshold and velocity interpretation: PDF pp. 24 and 68.
The first-crossing program finds the first stored sample meeting the comparison. The velocity quotient is an interval average, with strictly increasing times required for the stated chronological interpretation.

Laboratory assumptions: PDF pp. 73–75.
The ten readings and their unit are source data. The worked warning limit of 0.25 $\mathrm{m\,s^{-2}}$ and subtracted offset of 0.02 $\mathrm{m\,s^{-2}}$ are explicitly illustrative. Sampling times and assessment details are not supplied by the deck and are not invented here.

Worked debugging repairs: PDF p. 76.
The repairs explain that a singleton comma alone does not supply a second coordinate. The inconsistent-record repair retains the unit field rather than silently deleting it.

Added practice and answers.
The cumulative questions, independent test cases, illustrative electrical observations, and complete sensor report are study extensions of slide concepts. They do not create new course assessment requirements.

A.3. Slide-to-notes coverage map

Every source PDF page is mapped below. Page references in the right column point into *these reading notes*. The source-page count includes title and divider pages; the printed footer numbering in the slide deck differs at several points.

PDF page	Source content	Reading-note location
1	Title, course, instructor, institution, and term	Introduction, p. 2
2	Week 5 at a glance and organizing sequence	Introduction, p. 3
3	Lecture 13 divider	Chapter 1, p. 6
4	Lecture 13 learning goals	Chapter 1, p. 6
5	One sensor, several measurements	Section 1.1, p. 6
6	Why engineering data needs collections	Section 1.1, p. 6
7	A first Python list	Section 1.2, p. 7
8	A list is ordered	Section 1.3, p. 8
9	Indexing retrieves one element	Section 1.3, p. 8
10	Index versus value	Section 1.3, p. 8
11	Negative indices	Section 1.3, p. 8
12	Valid indices and list length	Section 1.3, p. 8
13	Invalid indexing and IndexError	Section 1.4, p. 9
14	Slicing a window	Section 1.5, p. 9
15	Excluded stop and predictable window length	Section 1.5, p. 9
16	Useful slice patterns	Section 1.6, p. 11
17	Prediction activity A–F	Section 1.7, p. 11

Continued on the next page

PDF page	Source content	Reading-note location
18	Length and membership	Section 1.8, p. 12
19	Built-in numerical summaries	Section 1.9, p. 13
20	Loop directly over values	Section 1.10, p. 14
21	Loop over indices and parallel lists	Section 1.11, p. 15
22	enumerate(): index and value	Section 1.12, p. 15
23	Choosing among three loop patterns	Section 1.12, p. 15
24	First threshold crossing	Section 1.13, p. 16
25	Units and meaning	Section 1.14, p. 17
26	Three collection mistakes	Section 1.15, p. 17
27	Lecture 13 exit ticket	Section 1.16, p. 18
28	Lecture 14 divider	Chapter 2, p. 19
29	Lecture 14 learning goals	Chapter 2, p. 19
30	Mutation warm-up trace	Section 2.1, p. 19
31	Meaning of mutability	Section 2.2, p. 20
32	Confirmed element replacement	Section 2.2, p. 20
33	In-place calibration	Section 2.3, p. 20
34	Build a processed calibration list	Section 2.4, p. 21
35	Building and filtering with append	Section 2.5, p. 22
36	append versus extend	Section 2.6, p. 23
37	Insertion at a chosen position	Section 2.7, p. 24
38	remove versus pop	Section 2.8, p. 24
39	Membership before remove	Section 2.8, p. 24
40	Deleting while iterating	Section 2.9, p. 25
41	Safe filtering	Section 2.10, p. 26
42	Concatenation and repetition	Section 2.11, p. 26
43	Mutation versus returned result	Section 2.12, p. 27
44	sort versus sorted	Section 2.13, p. 28
45	The None return from sort	Section 2.13, p. 28
46	Reversing in place or by slicing	Section 2.14, p. 29
47	Aliasing diagram	Section 2.15, p. 29
48	Aliasing in code	Section 2.15, p. 29
49	Independent outer copy	Section 2.16, p. 30
50	Mutation through a function argument	Section 2.17, p. 31
51	Traceable data-processing workflow	Section 2.18, p. 33
52	Lecture 14 exit ticket	Section 2.19, p. 33
53	Lecture 15 divider	Chapter 3, p. 34
54	Lecture 15 learning goals	Chapter 3, p. 34
55	Structure-choice warm-up	Section 3.1, p. 34
56	A first tuple	Section 3.2, p. 34
57	Singleton comma and empty tuple	Section 3.3, p. 35
58	Tuple sequence operations	Section 3.4, p. 36
59	Item assignment and immutability	Section 3.5, p. 36
60	Why choose immutable records	Section 3.5, p. 36
61	Packing and unpacking	Section 3.6, p. 37
62	Swapping bounds	Section 3.7, p. 38
63	Returning several related values	Section 3.8, p. 39
64	Documenting tuple field meanings	Section 3.9, p. 39
65	A list of calibration tuples	Section 3.10, p. 40
66	Unpacking records in a loop	Section 3.11, p. 41
67	Parallel lists versus paired records	Section 3.12, p. 41
68	Interval velocity from adjacent records	Section 3.13, p. 42
69	Shared list/tuple sequence interface	Section 3.14, p. 43
70	Choose by intended use	Section 3.14, p. 43
71	List/tuple conversion	Section 3.15, p. 43

Continued on the next page

PDF page	Source content	Reading-note location
72	Structure-selection activity	Section 3.16, p. 44
73	Laboratory 2 theme and source dataset	Section 3.17, p. 45
74	Proposed laboratory tasks	Section 3.17, p. 45
75	RMS pattern and formula	Section 3.18, p. 45
76	Four tuple/record debugging cases	Section 3.21, p. 52
77	Concept map and electrical-record exit ticket	Section 3.22, p. 54
78	Sequence quick reference	Section 4.2, p. 55
79	Operations and tuple-pattern quick reference	Section 4.3, p. 56
80	Suggested reading and preparation habit	Chapter A, p. 63
81	Closing message: preserve engineering meaning	Section 4.1, p. 55

A.4. Using the accompanying Python files

The companion script contains the successful executable examples and labelled demonstrations of logic mistakes. It does not execute the six listings intended to raise an exception. Those six remain in the PDF and are supplied as separate files in the source package's `examples/intentional_errors` folder.

Each code example can be restarted from its own stated input. The complete source package includes a verification utility and captured outputs. The utility checks the expected successful results and the expected exception classes separately. A test passing demonstrates the behaviour of that particular example; it does not prove that every possible dataset or physical interpretation is valid.

Key idea

A useful engineering collection preserves more than numbers. It preserves which numbers belong together, how they are ordered, which units they use, and what changes have been made. Keep those meanings visible from input to final report.